

Analisi Audio in Frequenza (pt.2)

Elaborazione dell'audio digitale

Ingegneria del Cinema, Informatica e Telecomunicazioni

Antonio Servetti

Internet Media Group

Dip. di Automatica ed Informatica

Politecnico di Torino

servetti@polito.it

<http://media.polito.it>

Sommario

- La Trasformata di Fourier Discreta (DFT)

TRASFORMATATA DI FOURIER *DISCRETA*

TRASFORMATA DI FOURIER *DISCRETA*

➔ ■ **Tipologie di Trasformate di Fourier**

- ✓ Serie, tempo continuo, tempo discreto, tempo e frequenza discreti
- Python – Trasformata di Fourier Discreta – FFT
- Finestratura ed effetto sulla trasformata
- ATTIVITA' – Dual Tone Multi Frequency

Riferimenti bibliografici (teoria)

■ De Poli, "Algorithms for Sound Music Computing"

- ✓ 1.4 Spectral analysis of discrete-time signals
 - 1.4.1 The discrete-time Fourier transform
- ✓ 1.5 Short-time Fourier analysis
 - 1.5.1 The Discrete Fourier Transform
 - 1.5.1.2 Resolution, leakage and zero-padding
 - 1.5.2 The Short-Time Fourier Transform
 - 1.5.2.2 Windowing and the uncertainty principle

*Appunti del corso di
Informatica Musicale
Univ. Padova (online)*

■ Proakis, "Digital Signal Processing"

- ✓ 4. Frequency Analysis of Signals
- ✓ 7. The Discrete Fourier Transform
 - 7.4 Frequency Analysis of Signals Using the DFT

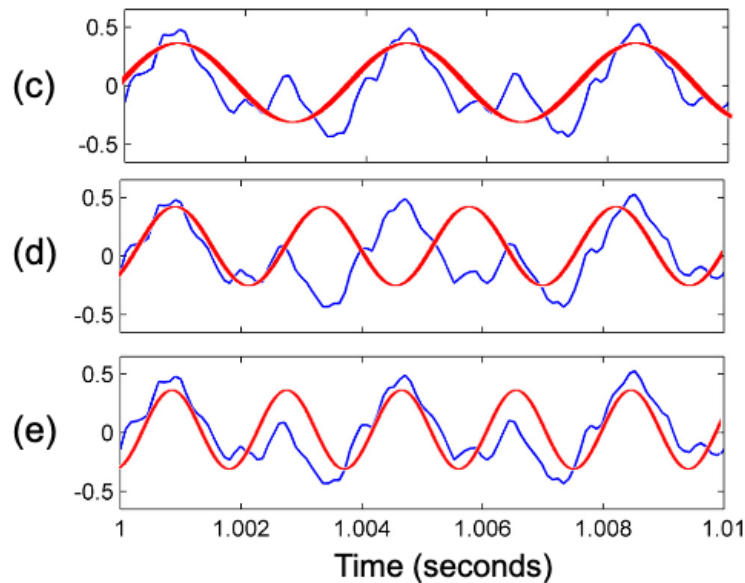
Riferimenti bibliografici (python)

- Muller, "Fundamentals of Music Processing",
 - ✓ Ch.2 Fourier Analysis of Signals
<https://www.audiolabs-erlangen.de/resources/MIR/FMP/C2/C2.html>

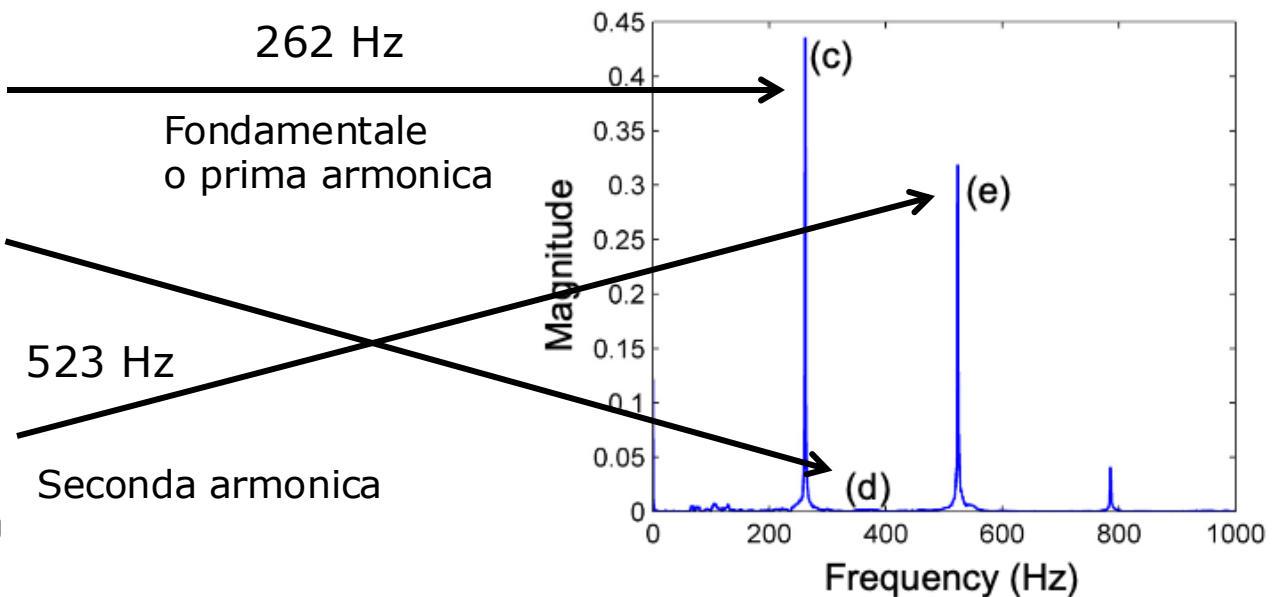
Analisi di Fourier - modulo

- L'analisi di Fourier è usata per estrarre l'informazione in frequenza
 - ✓ Il principio è quello di confrontare il segnale con sinusoidi a frequenza diversa e ottenere un indice di somiglianza (ampiezza/modulo) e un indice di disallineamento (fase)

CONFRONTO (somma / moltiplicazione)



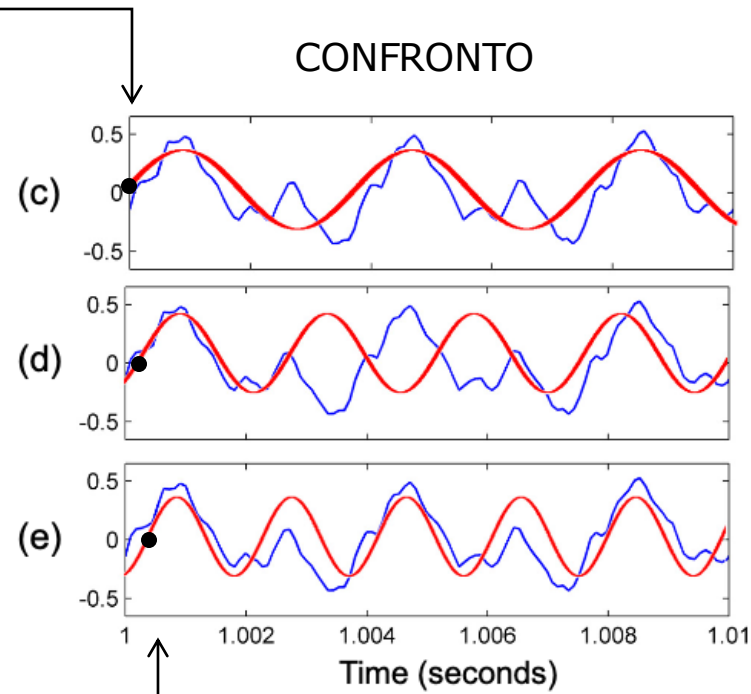
INDICE DI SOMIGLIANZA



Analisi di Fourier - fase

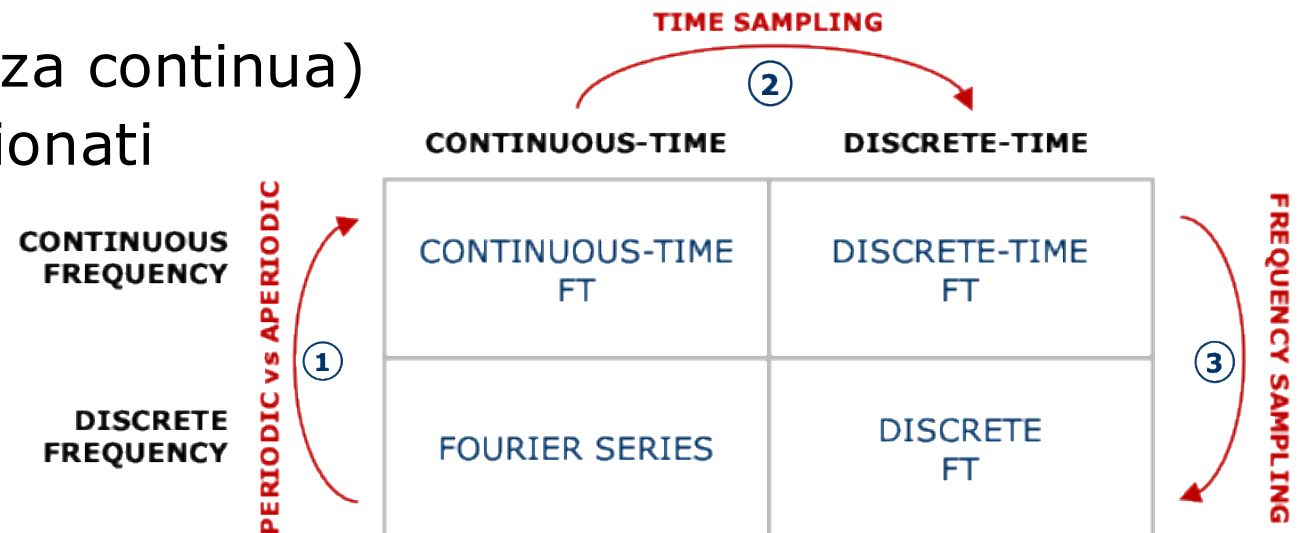
- Nell'analisi di Fourier la somiglianza tra le componenti in frequenza e il segnale è calcolata considerando anche lo shift temporale (disallineamento / fase) necessario per farle coincidere al meglio

INDICE DI DISALLINEAMENTO
e.g. posizione del primo 0
($\phi=0$) nel tempo



Tipi di analisi di Fourier

- Arriveremo alla definizione di Analisi di Fourier per segnali digitali discreti attraverso alcuni passaggi
 - ✓ Le serie di Fourier per l'analisi di segnali continui periodici
 - ✓ La trasformata di Fourier per l'analisi di segnali continui **a-periodici** ①
 - ✓ La trasformata di Fourier
 - ② **a tempo discreto** (e frequenza continua) per l'analisi di segnali campionati
 - ✓ La trasformata di Fourier discreta (DFT) per l'analisi di segnali digitali **a tempo**
 - ③ **digitali a tempo** e **frequenza discreta**



Reference: Proakis, "Principles of digital audio", Ch.4

Serie di Fourier - segnali periodici continui

- Segnali **periodici** continui possono essere descritti dalle serie di Fourier: somma pesata di sinusoidi *armoniche*

- ✓ Dove per armoniche si intende:
multiple *inter*e di una frequenza
fondamentale $F_0 = 1/T_p$ (dove T_p è il periodo)

$$x(t) = \sum_{k=-\infty}^{\infty} c_k e^{j2\pi k F_0 t}$$

- ✓ Per sinusoidi si intendono *esponenziali complessi*

- I coefficienti c_k della serie di Fourier sono calcolati come

- ✓ Dove l'integrale è calcolato su un periodo T_p del segnale

- ✓ I coefficienti di Fourier c_k sono valori complessi,
 - ma se il segnale è reale allora c_k e c_{-k} sono valori complessi coniugati: stesso modulo, fase opposta


$$c_k = \frac{1}{T_p} \int_{T_p} x(t) e^{-j2\pi k F_0 t} dt$$

Vedi condizioni di esistenza su: Proakis, "Digital Signal Processing", Ch.4 "Frequency Analysis of Signals"

Spettro di potenza - segnali periodici continui

- Un segnale periodico ha energia infinita, ma potenza media P_x finita

- ✓ P_x è la somma delle potenze medie di ciascuna armonica

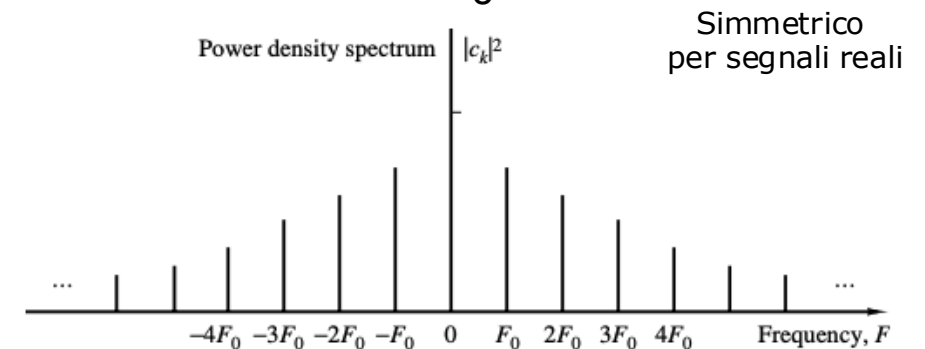
$$P_x = \frac{1}{T_p} \int_{T_p} |x(t)|^2 dt = \sum_{k=-\infty}^{\infty} |c_k|^2$$

- *Il diagramma di P_x per ciascuna frequenza $k \cdot F_0$ è detto densità spettrale di potenza o più semplicemente "spettro di potenza"*

- ✓ Consiste solo di frequenze discrete spaziate tra loro di F_0

- ✓ Possiamo anche rappresentare per ciascun c_k di ciascuna armonica

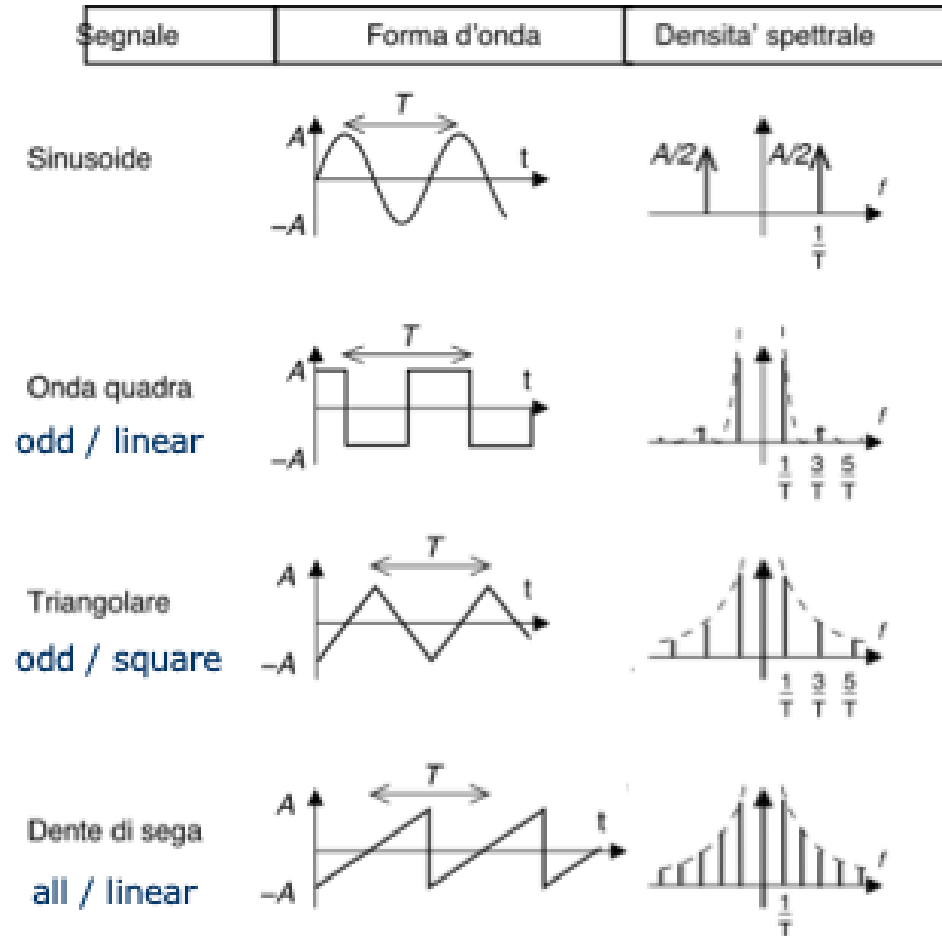
- Il modulo: spettro di ampiezza
- La fase: spettro di fase



Reference: Proakis, "Principles of digital audio", Ch.4 "Frequency Analysis of Signals"

Spettro di potenza - esempi

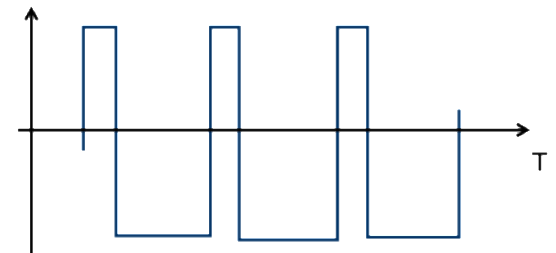
- Esempi di densità spettrale di segnali periodici continui noti



Reference: <https://thewolfound.com/sine-saw-square-triangle-pulse-basic-waveforms-in-synthesis/>

Treni di impulsi e duty-cycle

- Il treno di impulsi è una generalizzazione dell'onda quadra
- Definito il duty-cycle come il rapporto tra i due stati dell'onda
 - ✓ Dato un duty-cycle di $1:n$ (n intero) è mancante la n -esima armonica
 - ✓ Se n non è intero non mancano armoniche, ma il segnale è attenuato in quell'intorno
- Onda quadra, duty cycle $1:2$ 50%, mancano le armoniche pari
- Impulsi duty cycle $1:4$ 25%,
mancano le armoniche multiple di 4



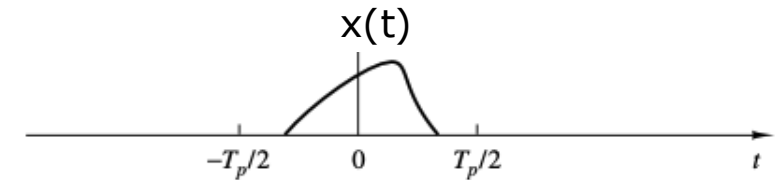
Ascolto: duties.mp3

Reference: <http://www.soundonsound.com/sos/feb00/articles/synthsecrets.htm>

Trasformata di Fourier – segnali aperiodici continui

- Premessa: nei segnali periodici l'inverso del periodo F_0 definiva la distanza delle componenti in frequenza, nel caso dei segnali a-periodici il periodo non esiste
- Assunzione: possiamo considerare un segnale a-periodico di durata finita x come se fosse un segnale periodico x_p con periodo T_p
 - ✓ Possiamo di conseguenza ottenere lo spettro di x dallo spettro di x_p

Importante: Trasformata di Fourier come rappresentazione di un segnale a-periodico (di durata finita) che viene periodicizzato



(a)

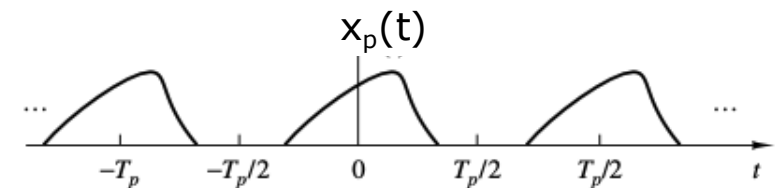


Figure 1.7
(a) Aperiodic signal $x(t)$ and (b) periodic signal $x_p(t)$ constructed by repeating $x(t)$ with a period T_p .

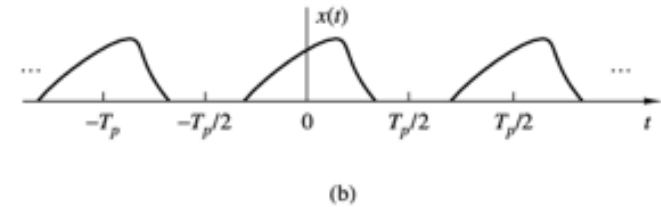
T_p

Trasformata di Fourier – segnali aperiodici continui

- Assumendo che un segnale a-periodico x (di durata finita) sia un segnale periodico x_p con periodo T_p tendente ad infinito
 - ✓ Possiamo sostituire x_p con x nell'espressione dei c_k di x_p ed integrare tra $-\infty$ e $+\infty$ (x è 0 al di fuori di $-T_p/2$ e $+T_p/2$)

$$c_k = \frac{1}{T_p} \int_{-T_p/2}^{T_p/2} x_p(t) e^{-j2\pi k F_0 t} dt \quad \Rightarrow \quad c_k = \frac{1}{T_p} \int_{-\infty}^{\infty} x(t) e^{-j2\pi k F_0 t} dt$$

- Con $T_p \rightarrow \infty$ allora $F_0 \rightarrow dF$ e kF_0 diventa la frequenza continua F e i c_k diventano valori (scalati $1/T_p$) della funzione continua $X(F)$ detta *Trasformata di Fourier*



$$X(F) = \int_{-\infty}^{\infty} x(t) e^{-j2\pi F t} dt$$

n.b. lo spettro è continuo

Spettro di energia - segnali aperiodici continui

- Un segnale aperiodico **di durata finita** ha energia finita che possiamo ricavare dalla sua Trasformata di Fourier

$$E_x = \int_{-\infty}^{\infty} |x(t)|^2 dt = \int_{-\infty}^{\infty} |X(F)|^2 dF \quad \text{Parseval's relation}$$

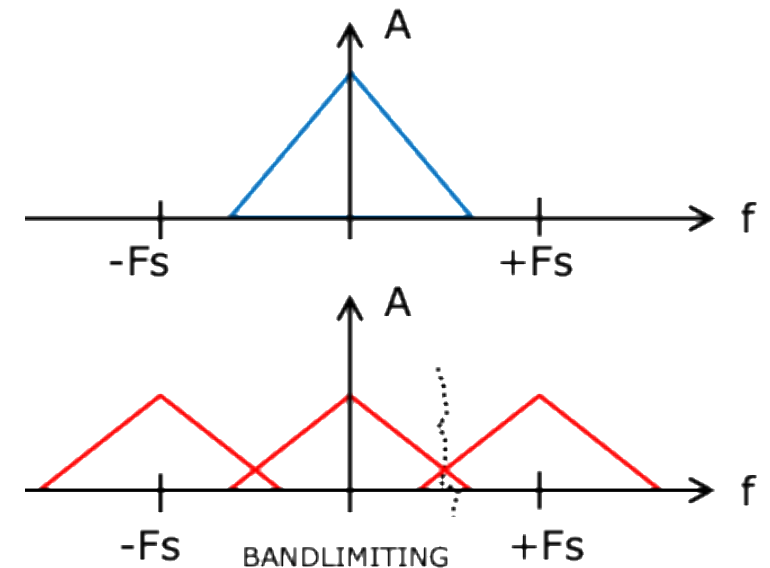
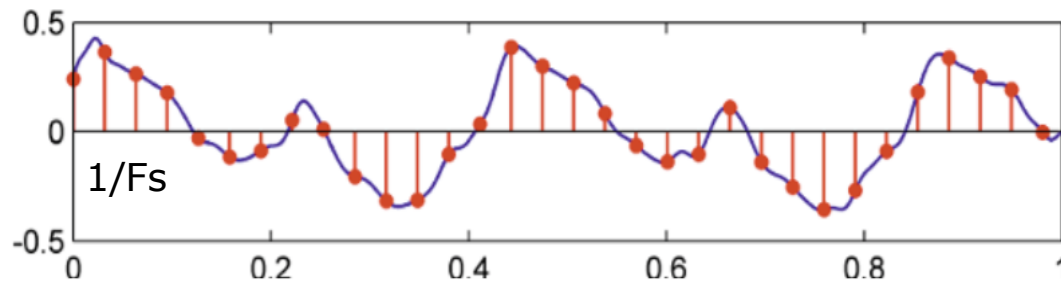
Ribadiamo il concetto di segnale aperiodico di durata finita. Su cui possiamo calcolare l'energia

- La quantità $S_{xx}(F) = |X(F)|^2$ "spettro di energia" (o densità spettrale di energia) rappresenta la distribuzione dell'energia del segnale in funzione della frequenza

Reference:

Spettro di un segnale campionato

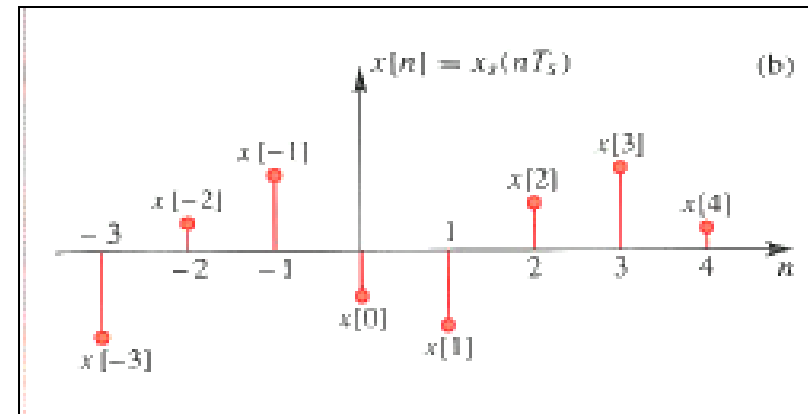
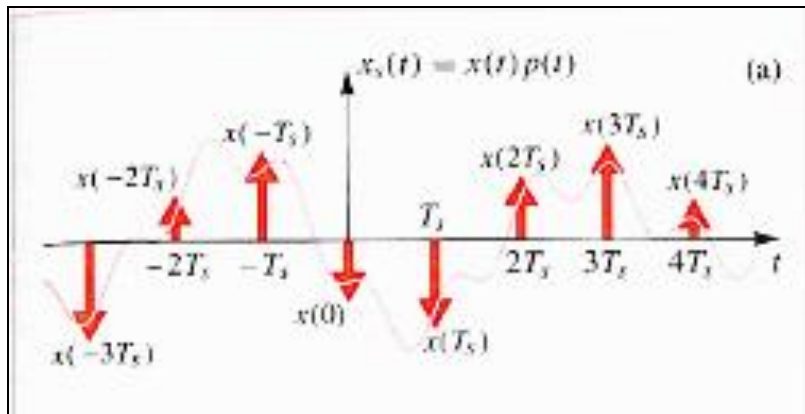
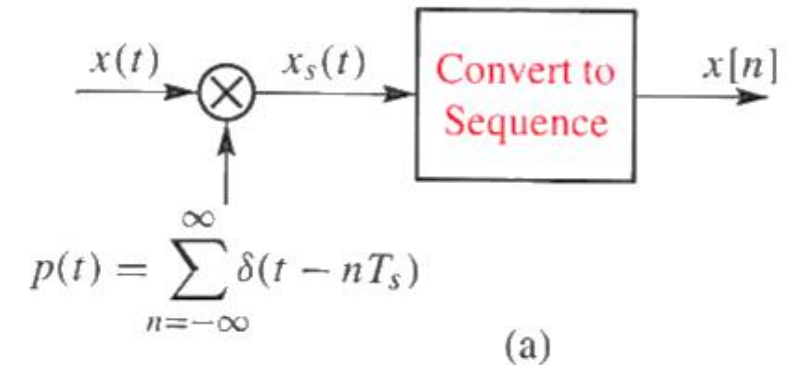
- L'operazione di campionamento produce un segnale con spettro PERIODICO dove lo spettro del segnale a tempo continuo è replicato a intervalli F_s (2π)
- Il campionamento di un segnale continuo necessita QUINDI dell'utilizzo di un filtro passa basso che elimini le frequenze al di sopra di $F_s/2$
 - ✓ $F_s/2$: frequenza di Nyquist,
 F_s : frequenza di campionamento



Spettro di un segnale campionato

- Il segnale campionato x_s si può vedere come il segnale continuo x moltiplicato per un treno di impulsi p

$$x_s(t) = x(t) p(t) = x(t) \sum_{n=-\infty}^{+\infty} \delta(t - nT_s)$$



Reference:

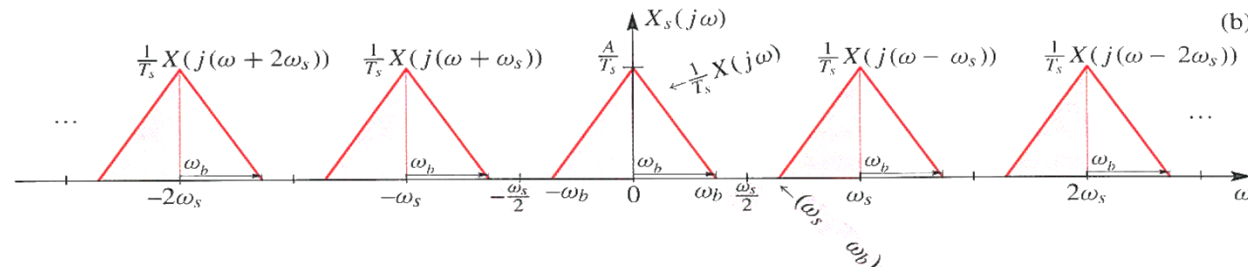
Spettro di un segnale campionato

- Dato x_s , il suo spettro è la convoluzione dello spettro di x con quello di p

$$x_s(t) = x(t) p(t) = x(t) \sum_{n=-\infty}^{+\infty} \delta(t - nT_s)$$
 - ✓ Lo spettro di un treno di impulsi a distanza T_s è un treno di delta a distanza ω_s , scalate di $1/T_s$

$$X_s(\omega) = X(\omega) * \sum_{n=-\infty}^{+\infty} \frac{1}{T_s} \delta(\omega - n\omega_s) = \frac{1}{T_s} \sum_{n=-\infty}^{+\infty} X(\omega - n\omega_s) \quad \omega_s = \frac{2\pi}{T_s}$$

- Lo spettro risultante X_s è equivalente allo spettro di x , X , ripetuto (e scalato) infinite volte nelle posizioni $n \cdot \omega_s$



Reference:

Trasformata di Fourier DT – segnali tempo discreto

- La Trasformata di Fourier di un segnale campionato è detta Trasformata di Fourier a Tempo Discreto (Discrete Time)

$$x_s(t) = x(t) p(t) = x(t) \sum_{n=-\infty}^{+\infty} \delta(t - nT_s)$$

- Possiamo calcolarla come la trasformata di x_s

$$F_{\omega}\{x_s(t)\} = F\left\{x(t) \sum_{n=-\infty}^{+\infty} \delta(t - nT_s)\right\} = F\left\{\sum_{n=-\infty}^{+\infty} x(nT_s) \delta(t - nT_s)\right\} \quad \dots x(nT_s) \text{ non dipende da } t$$

$$= \sum_{n=-\infty}^{+\infty} x(nT_s) F\{\delta(t - nT_s)\} = \sum_{n=-\infty}^{+\infty} x(nT_s) e^{-j\omega(nT_s)}$$

Tempo discreto: nT_s
Frequenza continua: ω

La trasformata di un impulso traslato nel tempo
è un esponenziale complesso con una corrispondente rotazione di fase

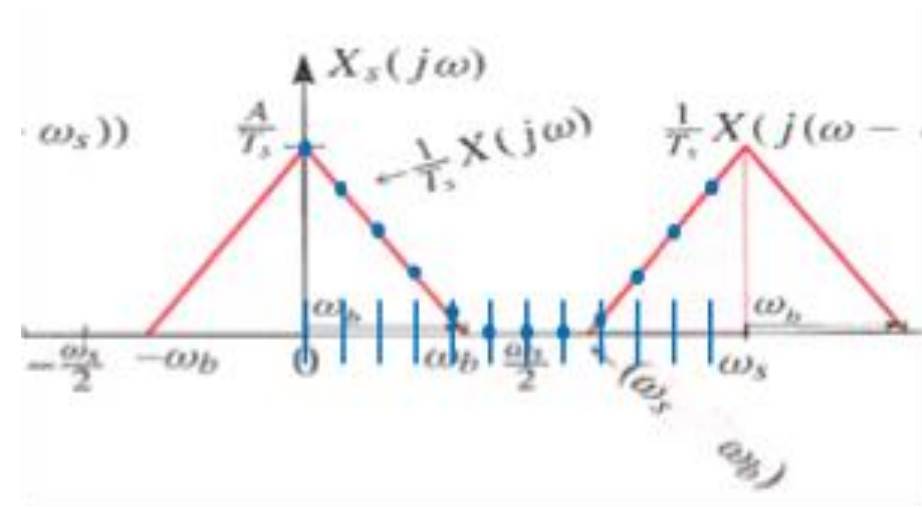
Reference:

Trasformata di Fourier Discreta – segnali discreti

- La Trasformata di Fourier Discreta (DFT)
è derivata dalla DTFT campionata ad intervalli regolari
 - ✓ La DTFT viene valutata (solo) su un periodo (altrimenti infinita)
 - Le repliche sono identiche
 - ✓ Su un periodo, e.g. da 0 a ω_s , la DTFT viene valutata in N punti equidistanti ω_k (insieme discreto di frequenze)

$$\omega_k = \frac{2\pi}{N}k \text{ for } k = 0, 1, \dots, N-1$$

$$F_{\omega_k}\{x_s(t)\} = \sum_{n=-\infty}^{+\infty} x(nT_s)e^{-j\omega_k(nT_s)}$$



TRASFORMATA DI FOURIER *DISCRETA*

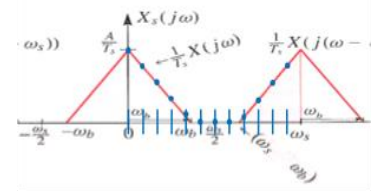
- Tipologie di Trasformate di Fourier
 - ✓ Serie, tempo continuo, tempo discreto, tempo e frequenza discreti
- ➔ ■ **Python – Trasformata di Fourier Discreta – FFT**
- Finestratura ed effetto sulla trasformata
- ATTIVITA' – Dual Tone Multi Frequency

Python DFT (via numpy FFT)

- In Python/NumPy la DFT è implementata tramite il modulo `np.fft`
 - ✓ Dettagli implementativi: <https://numpy.org/doc/stable/reference/routines.fft.html#implementation-details>
- `X = np.fft.fft(x, n=, norm=)` => `np.fft.rfft`
 - ✓ `x`: segnale su cui calcolare la DFT
 - ✓ `n`: numero di punti in frequenza su cui è calcolata la DFT
se non espresso equivale a `len(x)`
 - ✓ `norm`: criterio di normalizzazione – non ci interessa
e.g. divisione $1/N$ backward, forward o ortho ($1/\sqrt{N}$)
 - ✓ `X`: gli `n` valori COMPLESSI, c_k , della trasformata
- `F = np.fft.fttfreq(n, d=1/Fs)` => `np.fft.rfftfreq`
 - ✓ Ritorna i valori in frequenza su cui è stata calcolata la DFT

Reference: <https://numpy.org/doc/stable/reference/routines.fft.html>

Python DFT (via numpy FFT)



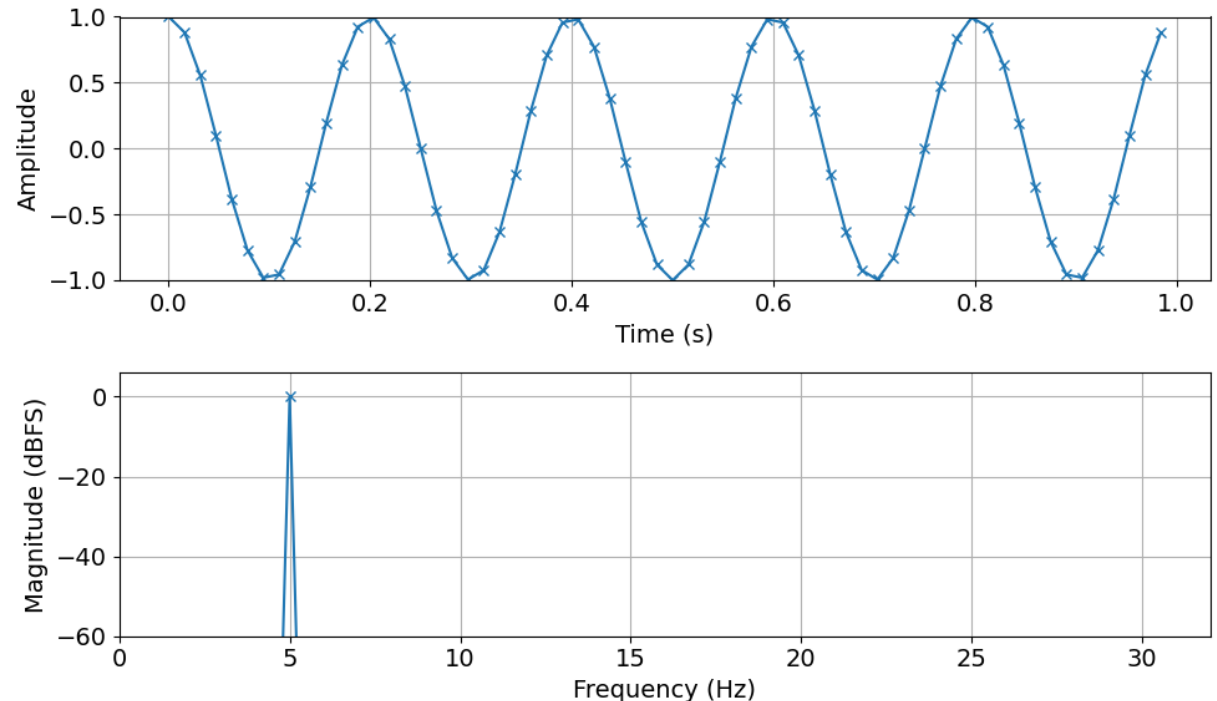
■ Esercizio (trial and error)

- ✓ Generare una sinusoide
x: A: 1, f: 5 Hz, Fs=64,
durata 1s

■ Attività

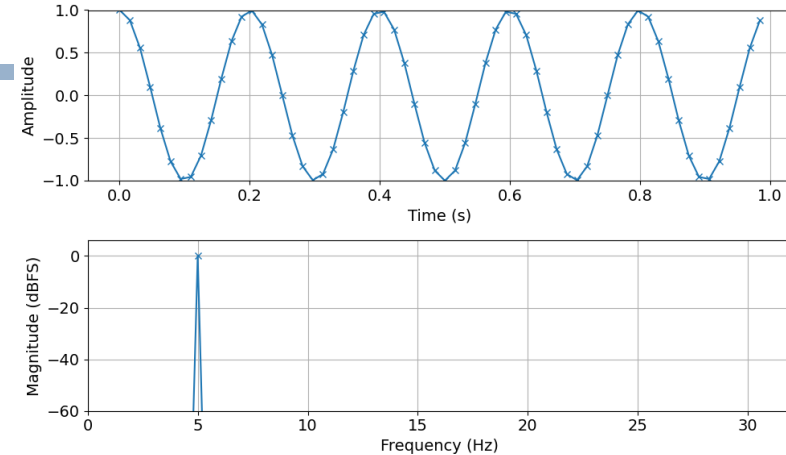
- ✓ Ottenere i grafici a lato

- Utilizzare `fftfreq` per ottenere l'asse delle x
- Calcolare il modulo in dB dello spettro normalizzato così che la sinusoide sia rappresentata a 0 dBFS (indipendentemente dalla sua durata)



Python DFT (via numpy FFT)

- Data la sinusoide x :
A: 1, f: 5 Hz, $F_s=64$, dur: 1s
- L'FFT ritorna i coefficienti c_k complessi
 - ✓ Occorre ottenerne il modulo $X_ = \text{np.abs}(X)$
 - ✓ Calcolarne il valore in dB: $X_ \text{dBFS} = 20 * \text{np.log10}(X_)$
- In questo modo però il valore è funzione del numero di campioni N
- Per ottenere un valore indipendente da N
 - ✓ Si divide X per N
- Per ottenere il corretto valore in dBFS
 - ✓ Si moltiplica per 2

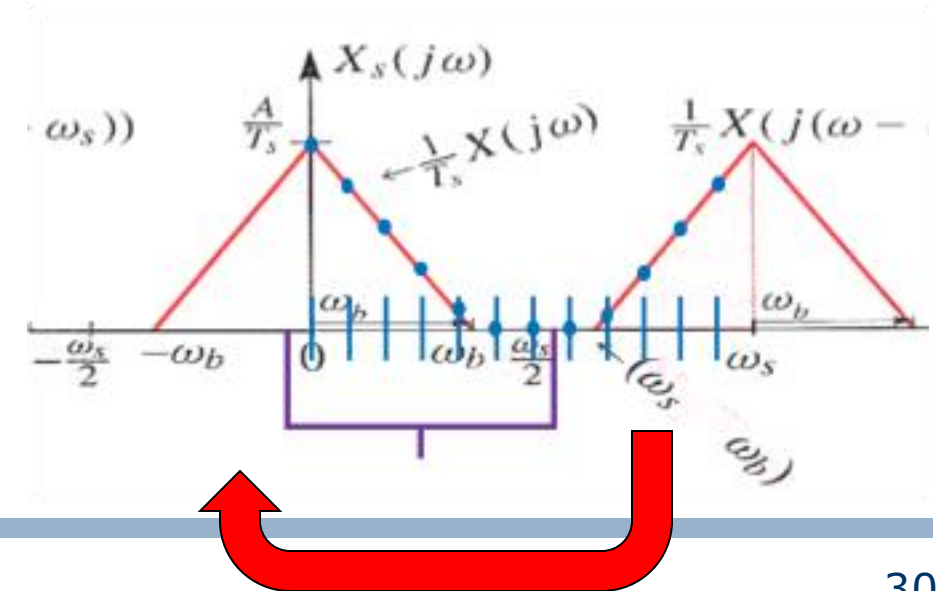


$$X_ = \text{np.abs}(X) / (N/2)$$

Python DFT – segnali reali

- E' noto che la DFT di segnali reali è simmetrica
 - ✓ Simmetria pari per il modulo e dispari per la fase
- E' quindi sufficiente analizzarne (circa) metà
 - ✓ Nel caso di N pari da $X[0]$ a $X[N//2]$ compreso
 - ✓ Quindi $N//2+1$ valori
- Analizziamo il vettore F ritornato da `rfftfreq` (e.g., $F_s=64$, $N=64$)
 - ✓ $F[0] = 0$, $F[1] = 1$, ...
 $F[5] = 5$, $F[6] = 6$

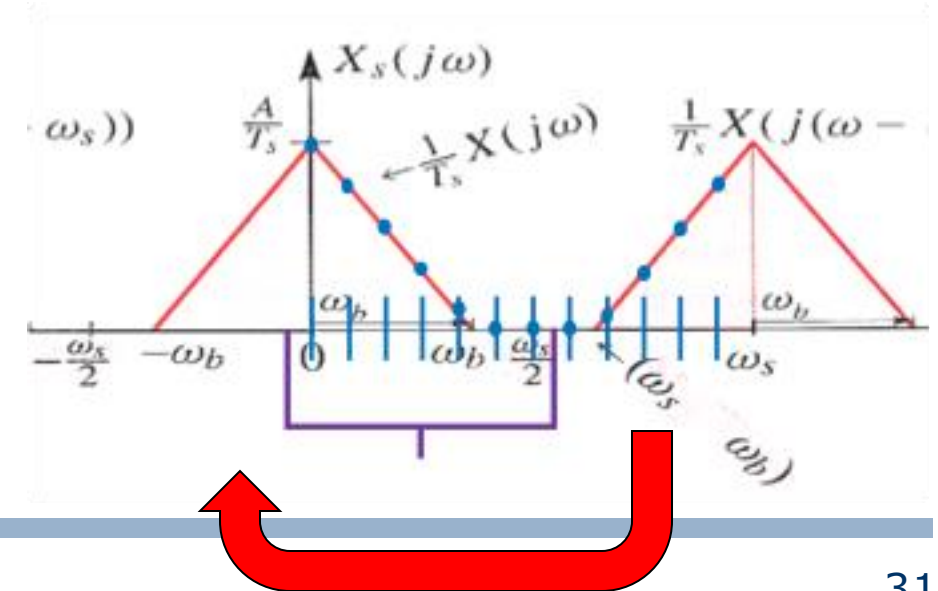
Esempio di prima



Si vedano anche le funzioni "reali": rfft e rfftfreq

DFT: risoluzione in frequenza teorica

- Dato un segnale di lunghezza N campioni
- La DFT ritorna, di default, N valori in frequenza campionati sullo spettro continuo a distanza $\Delta F = F_s/N$
- Dato lo spettro tra 0 ed F_s diviso in N porzioni le frequenze sono $F_k = k * F_s/N$, con $k=0:1:N$
- Il valore ΔF è detto risoluzione in frequenza "teorica"
quando $N = L$



Python DFT – segnali reali (rfft)

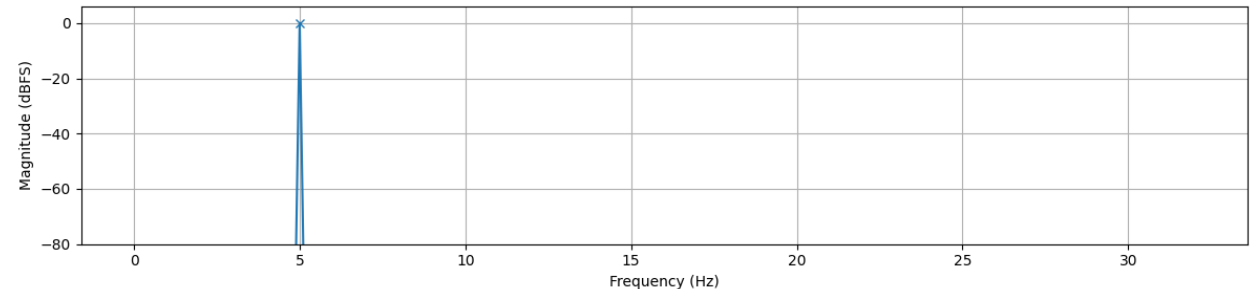
■ Normalizzazione

- ✓ Tono $A=1 \Rightarrow 0$ dB
- ✓ Fattore $1 / (L/2)$

■ FFT reale

- ✓ Ritorna $N/2+1$ elementi da 0 a $F_s/2$

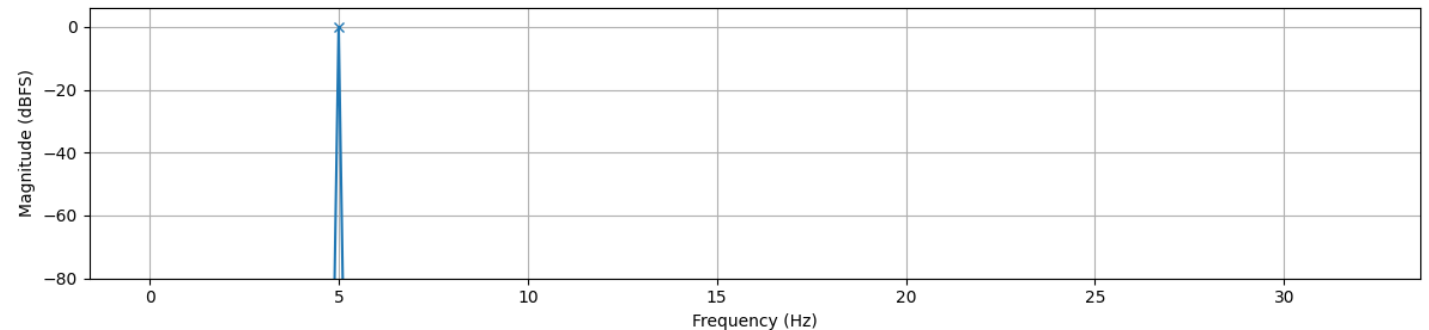
```
def dft(x, Fs, N=None):  
    L = len(x)  
    N = L if N is None else N  
    X = np.fft.rfft(x, N)  
    X_ = np.abs(X) / (L/2)  # Normalizzazione  
    freq = np.fft.rfftfreq(N, d=1./Fs)  
    # X_ = X_[:N//2+1]  
    # freq = freq[:N//2+1]  
    # X_dBFS = 20 * np.log10(X_)  
    return X_, freq
```



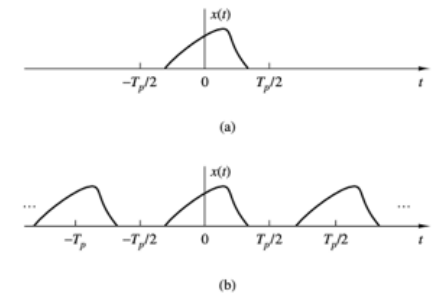
Python Numpy DFT – segnali reali (rfft)

```
# plot frequency spectrum
def plot_vs_freq(X, f, Fs, type="linear", xlim=None, ylim=None, fig=None, zp=False, fmt="-"):
    fig, ax = plt.subplots(figsize=(6,3)) if fig is None else fig
    xlim = [np.min(f), np.max(f)] if xlim is None else xlim
    if type == "linear":
        ax.set_ylabel('Magnitude')
        ylim = [0, 1.2] if ylim is None else ylim
        X = X
    elif type == "log":
        ax.set_ylabel('Power (dBFS)')
        ylim = [-60, +6] if ylim is None else ylim
        X = 10*np.log10(X**2)
    ax.plot(f, X, fmt)
    ax.set_xlabel('Frequency (Hz)')
    ax.set_xlim(xlim)
    ax.set_ylim(ylim)
    plt.grid(visible=True);
    plt.tight_layout()

plot_vs_freq(X, f, Fs)
```

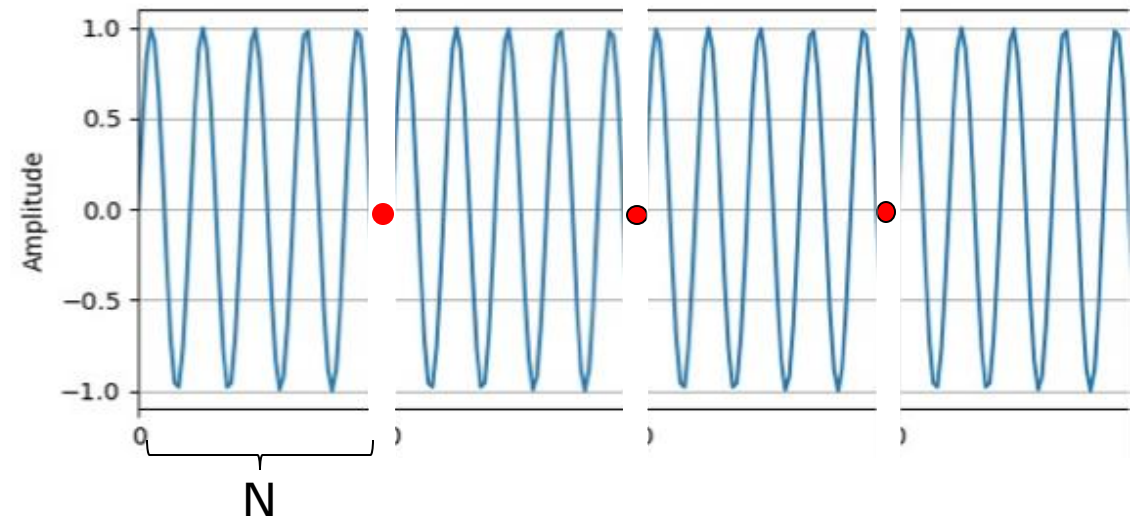


Effetti collaterali nella DFT



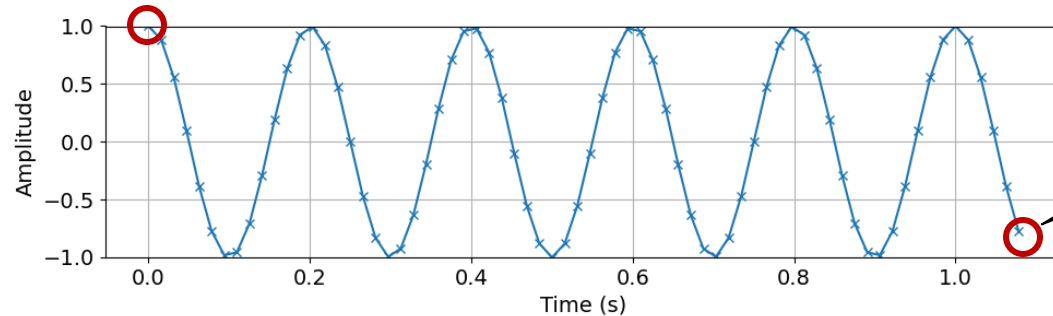
- Abbiamo introdotto la DFT come estensione della Trasformata di Fourier quindi supponendo di avere segnali periodici (o a-periodici di durata finita, ma periodicizzati)
- Quando calcoliamo la DFT non possiamo calcolarla se non su un numero finito di campioni del segnale N (detto "finestra")
- La DFT assume che questi N siano un periodo del nostro segnale, ripetuto all'infinito
 - ✓ Cosa succede se c'è una DISCONTINUITA' tra il primo e l'ultimo campione degli N ?

In questo caso il segnale è (sembra) periodico, ma non c'è un numero intero di periodi nella finestra



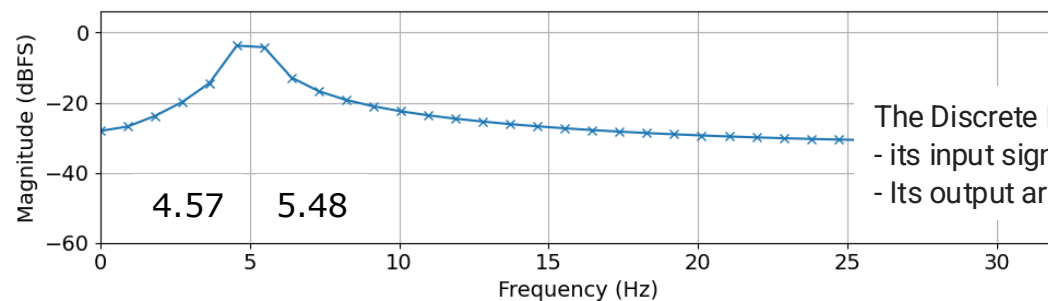
Effetti collaterali nella DFT

- L'effetto collaterale della discontinuità nella periodizzazione del nostro segnale finestrato / troncato è che la DFT è la versione "sporca" della FT del segnale "ideale"
- Esempio di senoide A: 1, f: 5 Hz, $F_s=64$, dur: 1.1s



Adesso **N=70**

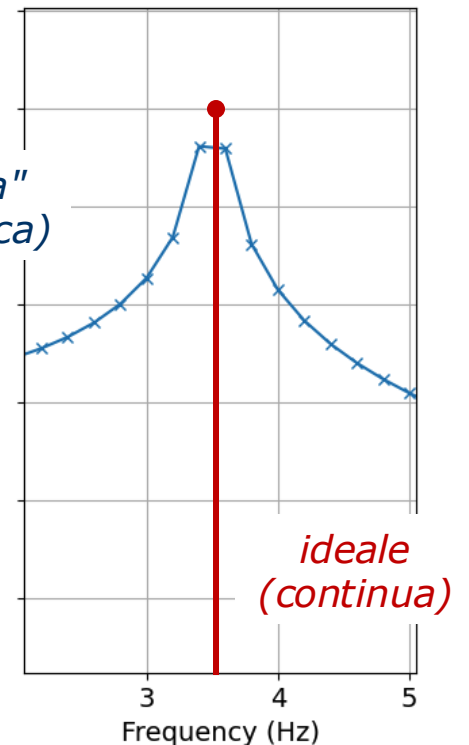
$$\Delta F = \frac{F_s}{N} = \frac{64}{70} = 0.91 \text{ Hz}$$



The Discrete Fourier Transform (DFT) assumes that

- its input signal is one period of a periodic signal.
- Its output are the discrete frequencies of this periodic signal.

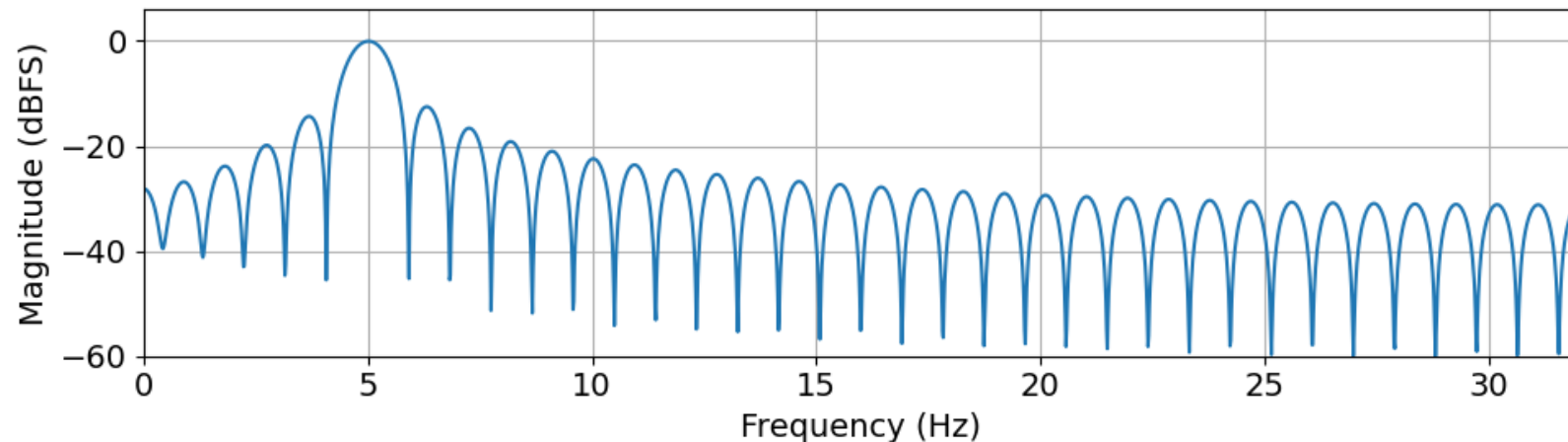
"sporca"
(numerica)



Reference: <https://dspillustrations.com/pages/posts/misc/spectral-leakage-zero-padding-and-frequency-resolution.html>

Dispersione spettrale (spectral leakage)

- Il segnale periodicizzato della sinusoide troncata e ripetuta non è una vera sinusoide, la transizione brusca tra un periodo e il successivo introduce componenti in frequenza aggiuntive.
- Il fenomeno è chiamato **spectral leakage** (dispersione spettrale)
 - ✓ n.b. nell'esempio precedente, $f=5$, la porzione periodicizzata conteneva un multiplo intero del periodo e il fenomeno non si "vedeva"

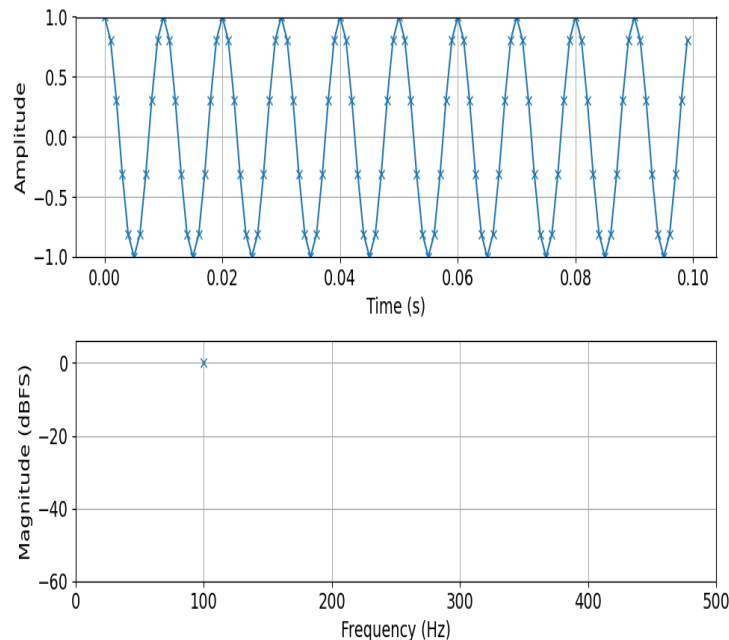


DTFT
E' una fz continua

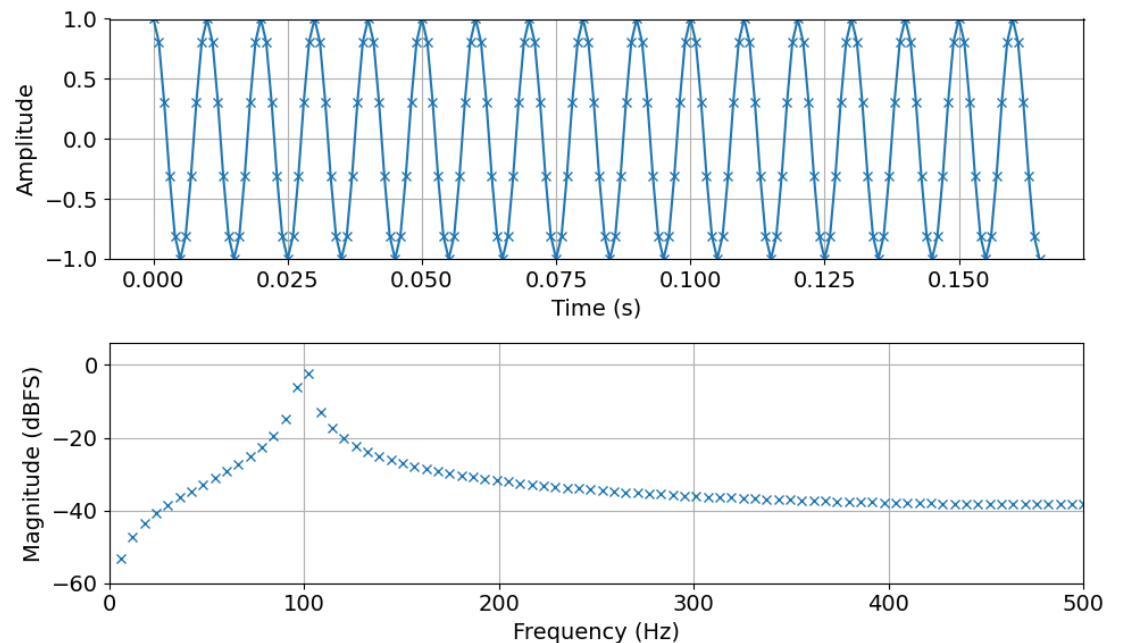
Ascoltare lo spettro del segnale troncato

- Il nostro orecchio fa una analisi in frequenza (come la DFT)
- Come suona la periodicizzazione di un segnale audio troncato?
- Esempio: senoide a 100 Hz ed F_s 1000

1/10 di secondo, numero intero di periodi



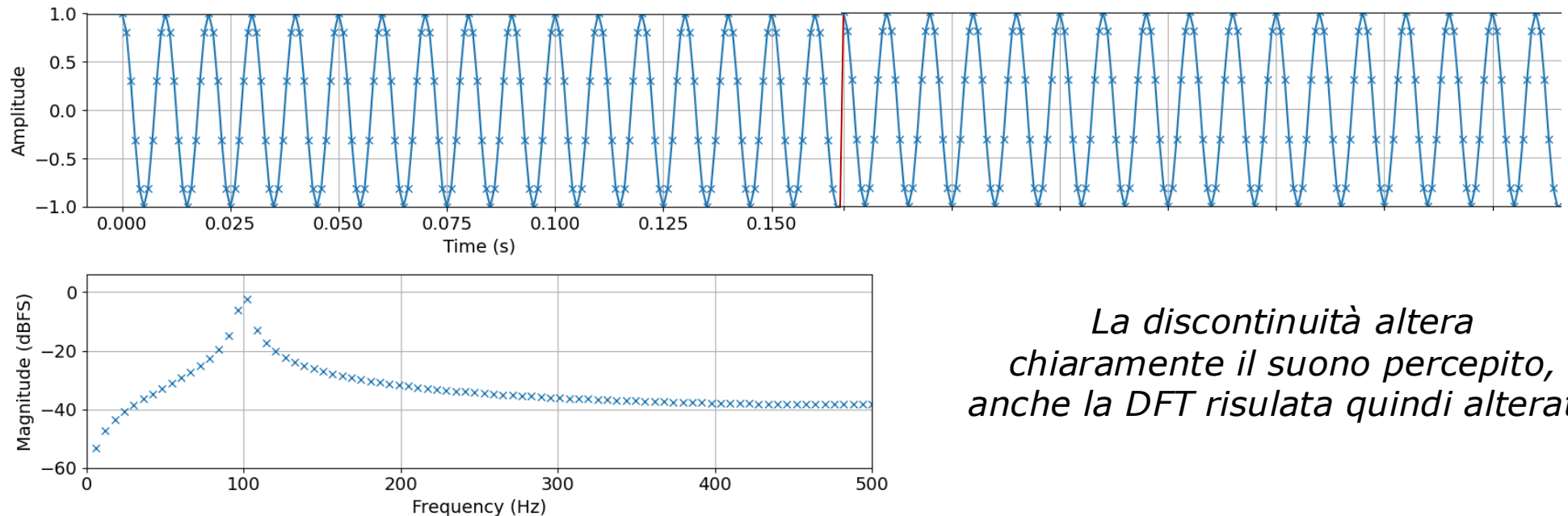
1/6 di secondo, numero NON intero di periodi



Ascoltare lo spettro del segnale troncato

- Il nostro orecchio fa una analisi in frequenza (come la DFT)
- Come suona la periodicizzazione di un segnale audio troncato?
- Esempio: senoide a 100 Hz ed Fs 1000

1/6 di secondo, numero NON intero di periodi



*La discontinuità altera
chiaramente il suono percepito,
anche la DFT risultata quindi alterata*

Zero Padding : campionare lo spettro più frequentemente

- Dal momento che la DFT calcola un numero di valori in frequenza pari al numero di campioni nel tempo
 - ✓ Un "trucco" per rendere più fitto il campionamento dello spettro è quello di aggiungere zeri "allungando" in modo fittizio il segnale
 - ✓ Di solito si sceglie una lunghezza totale (N_{fft}) che è potenza di 2
- In questo caso occorre prestare attenzione che
 - ✓ NON AUMENTA la risoluzione in frequenza teorica $\Delta F = F_s/L$
 - ✓ Si introduce una risoluzione in frequenza apparente (fittizia) pari a $\Delta F_a = F_s/N_{fft}$
- Aggiungendo valori nulli lo spettro del segnale "allungato" è equivalente a quello originale

*Uso L ed N_{fft} per
per esplicitare la
dipendenza*

Zero Padding : campionare lo spettro più frequentemente

■ Esempio: segnale con tre sinusoidi, 16 campioni

- ✓ (a) DTFT
- ✓ (b) DFT senza zero padding (9 punti)
 - Non è possibile identificare correttamente i tre toni
- ✓ (c) DFT con $N_{fft} = 32$
 - E' già possibile identificare i tre toni e le loro frequenze
- ✓ (d) DFT con $N_{fft} = 128$

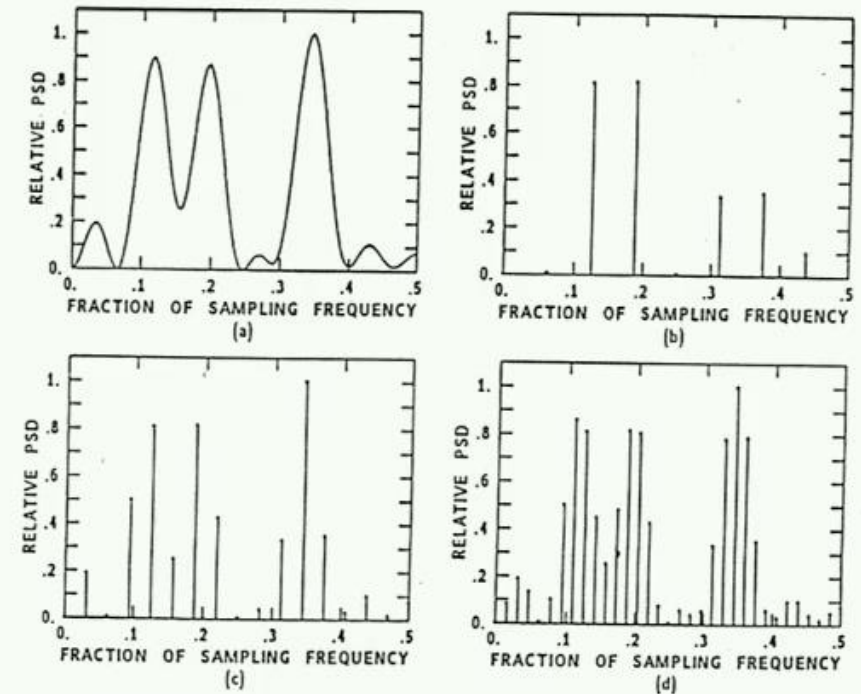


Figure 2.3 Transform interpolation by zero padding. (a) Magnitude of DTFT for a 16-point data record containing three sinusoids. (b) Magnitude of DTFS with no zero padding. Ambiguities are unresolved. (c) Magnitude of DTFS with double padding. Ambiguities are resolved because the three sinusoids can be distinguished. (d) Magnitude of DTFS with eight times zero padding.

Zero Padding : campionare lo spettro più frequentemente

■ `np.fft.fft(x, n)`

- ✓ `n`: intero, opzionale - numero di valori in output
 - se minore della lunghezza dell'input, l'input è troncato,
 - se maggiore, vengono aggiunti degli zeri all'input
 - se assente viene utilizzato l'input così come è

```
def dft(x, Fs, N=None):  
    L = len(x)  
    N = L if N is None else N  
    X = np.fft.rfft(x, N)  
    X_ = np.abs(X) / (L/2)  
    freq = np.fft.rfftfreq(N, d=1./Fs)  
    # X_ = X_[:N//2+1]  
    # freq = freq[:N//2+1]  
    # X_dBFS = 20 * np.log10(X_)  
    return X_, freq
```

`numpy.fft.fft`

`fft.fft(a, n=None, axis=-1, norm=None, out=None)` [\[source\]](#)

Compute the one-dimensional discrete Fourier Transform.

This function computes the one-dimensional n -point discrete Fourier Transform (DFT) with the efficient Fast Fourier Transform (FFT) algorithm [CT].

Parameters:

`a` : *array_like*

Input array, can be complex.

`n` : *int, optional*

Length of the transformed axis of the output. If n is smaller than the length of the input, the input is cropped. If it is larger, the input is padded with zeros. If n is not given, the length of the input along the axis specified by `axis` is used.

TRASFORMATA DI FOURIER *DISCRETA*

- Tipologie di Trasformate di Fourier
 - ✓ Serie, tempo continuo, tempo discreto, tempo e frequenza discreti
- Python – Trasformata di Fourier Discreta – FFT
- ➔ ■ **Finestratura ed effetto sulla trasformata**
- ATTIVITA' – Dual Tone Multi Frequency

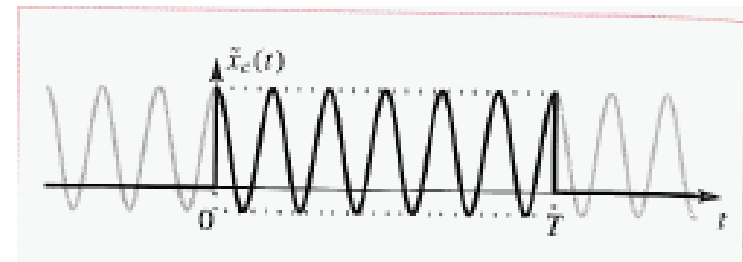
Finestratura

- Per calcolare la DFT è necessario utilizzare un segnale di lunghezza finita (L campioni)
- Estrarre L campioni da un segnale $x[n]$ equivale a "finestrare" il segnale:
 - ✓ cioè moltiplicare (implicitamente) il segnale per una *finestra rettangolare* di L campioni definita come segue

Finestra rettangolare (boxcar)

$$w[n] = \begin{cases} 1 & 0 \leq n \leq L - 1 \\ 0 & \text{altrimenti} \end{cases}$$

$$x[n] = 0.5 + 0.8 \cos(\omega n)$$



$$x_w[n] = x[n] \cdot w[n]$$

Effetto in frequenza della finestatura

- Calcolare la DFT di un segnale troncato $x_w[n]$ è invero la DFT del segnale finestrato $x_w[n] = x[n] \cdot w[n]$ non la DFT di $x[n]$
- La sua trasformata di Fourier è per definizione la convoluzione delle trasformate dei due segnali
 - ✓ Moltiplicazione nel tempo corrisponde alla convoluzione in frequenza

$$X_w[k] = X[k] * W[k]$$

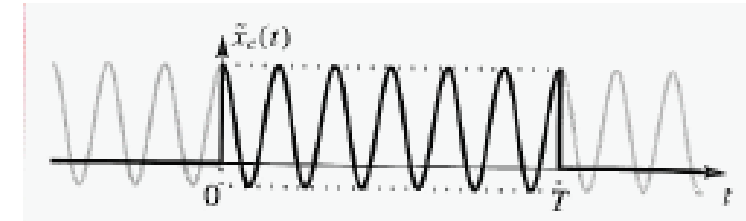
- Quindi lo spettro del segnale finestrato è lo spettro "vero" del segnale infinito, distorto (smussato/spalmato) dalla risposta in frequenza della finestra.

Reference:

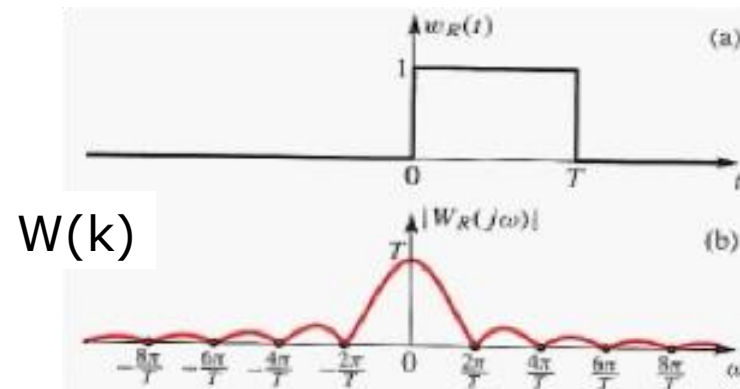
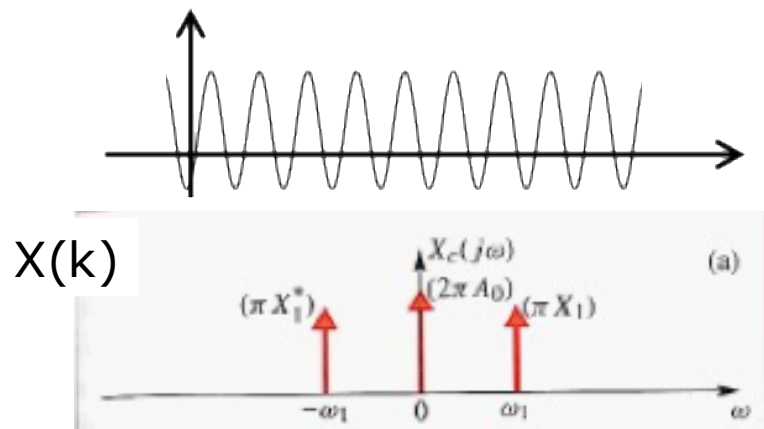
Effetto in frequenza della finestrazione

- Consideriamo l'esempio precedente

$$x(t) = 0.5 + 0.8 \cos(\omega t) \quad w(t) = \begin{cases} 1 & 0 \leq t \leq T \\ 0 & \text{otherwise} \end{cases}$$



- Le rispettive trasformate saranno come segue

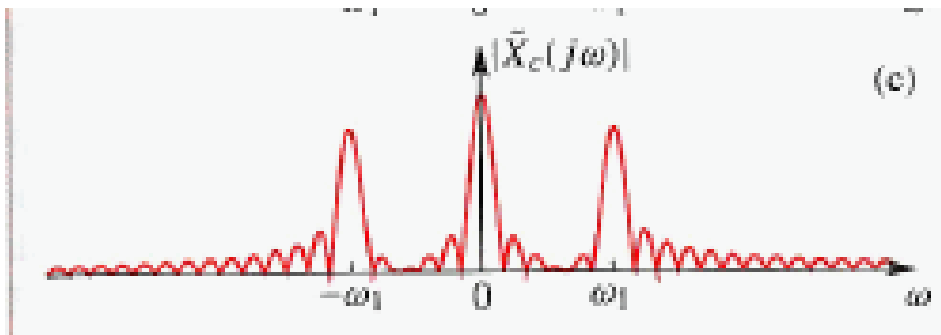
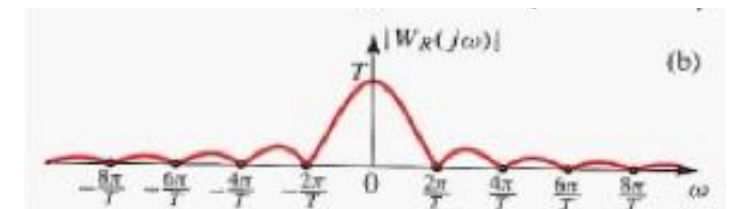
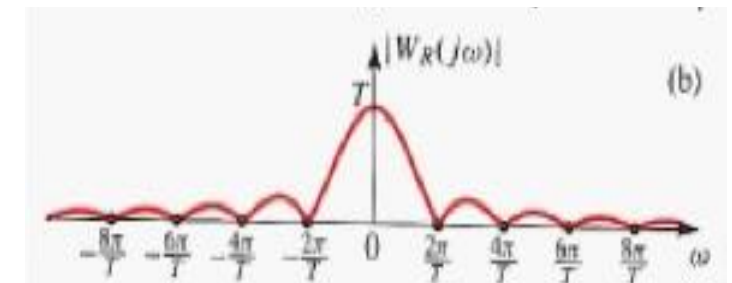
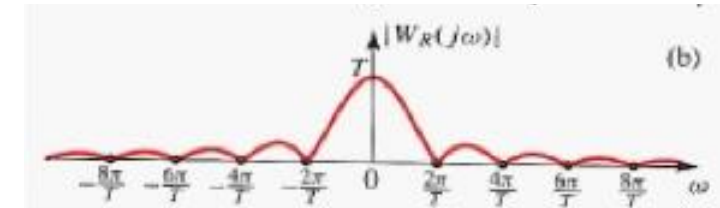
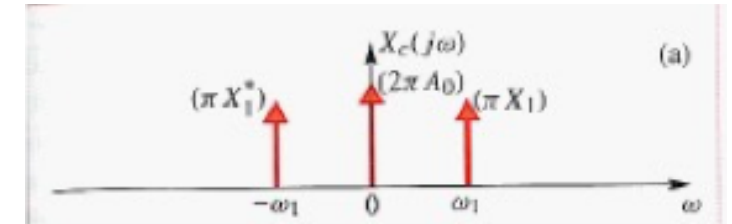


$$\left| \text{sinc}(f) = \frac{\sin(\pi f T)}{\pi f T} \right|$$

Domanda: Quale è la loro convoluzione?

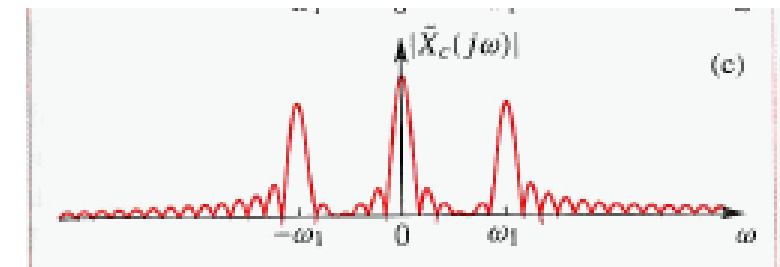
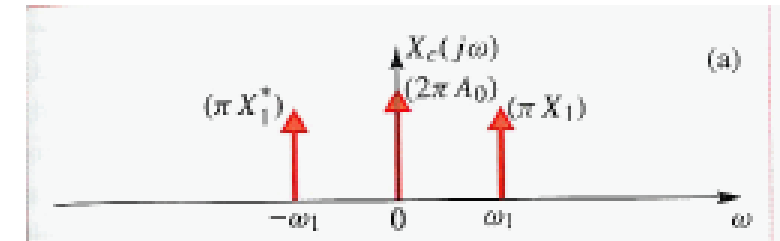
Convoluzione

- La convoluzione delle due trasformate corrisponde
 - ✓ Corrisponde alla somma
 - ✓ Della moltiplicazione della sinc (traslata)
 - ✓ Per ogni delta della trasformata del segnale
 - Cioè per l'energia di ogni componente in frequenza



Effetto in frequenza della finestrazione

- Analizziamo l'effetto in frequenza dovuto alla finestrazione
- Per ogni componente in frequenza dello spettro $\omega > 0$
 - ✓ Il suo valore viene moltiplicato per la *sinc*
- La *sinc* è costituita da
 - ✓ Una campana principale (main lobe)
 - Che "spalma" sulle frequenze adiacenti il valore della componente – smearing (*sbavatura*)
 - ✓ Dei lobi laterali (side lobes)
 - Che introducono del rumore in porzioni dello spettro che sarebbero state uguali a zero – leakage (*dispersione*)

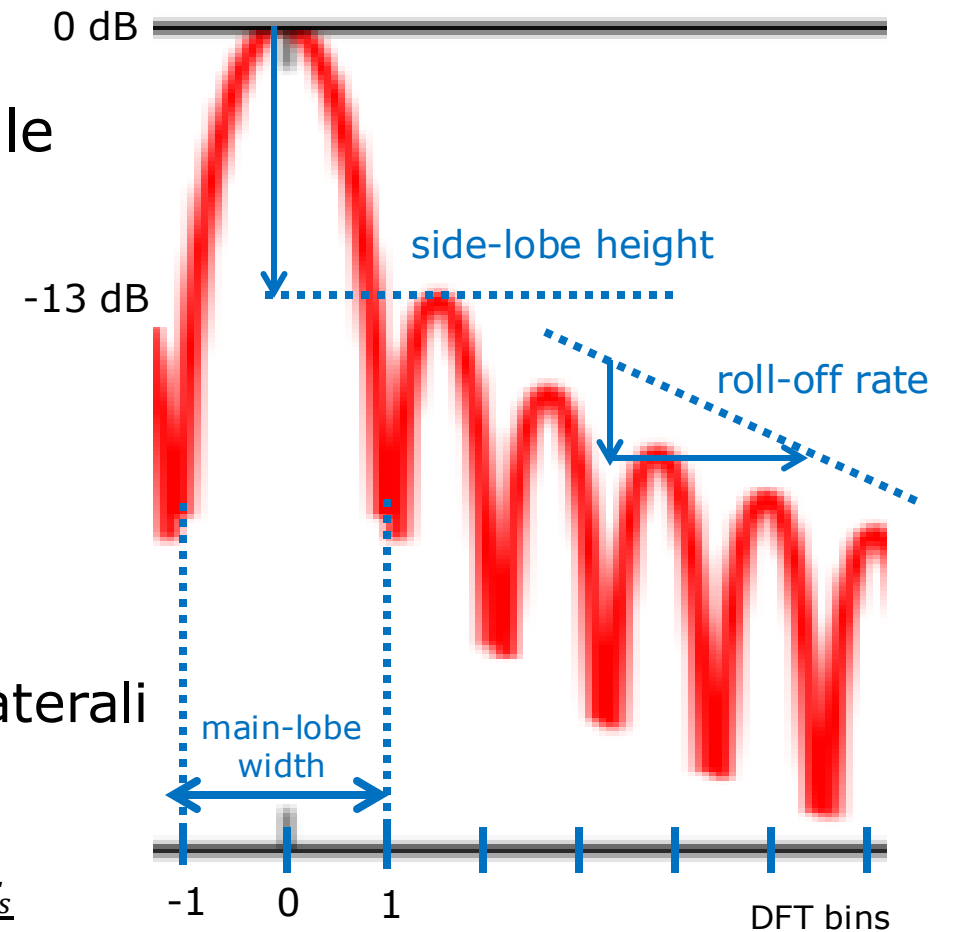


Spettro della finestra

- Gli spettri delle finestre possono essere caratterizzati da tre proprietà principali

- ✓ Larghezza della campana/lobo principale
 - Dove si annulla (0) o $-\infty$ (dB)
- ✓ Altezza del primo lobo laterale
 - Differenza in dB rispetto all'altezza del lobo principale
- ✓ Tasso di decadimento (roll-off rate) dei lobi laterali
 - Tasso di riduzione dell'altezza dei lobi laterali per ogni decade (*10) o ottava (*2) di frequenza

Larghezza di un bin in frequenza: $\Delta F = \frac{2\pi}{T} = \frac{F_s}{L}$

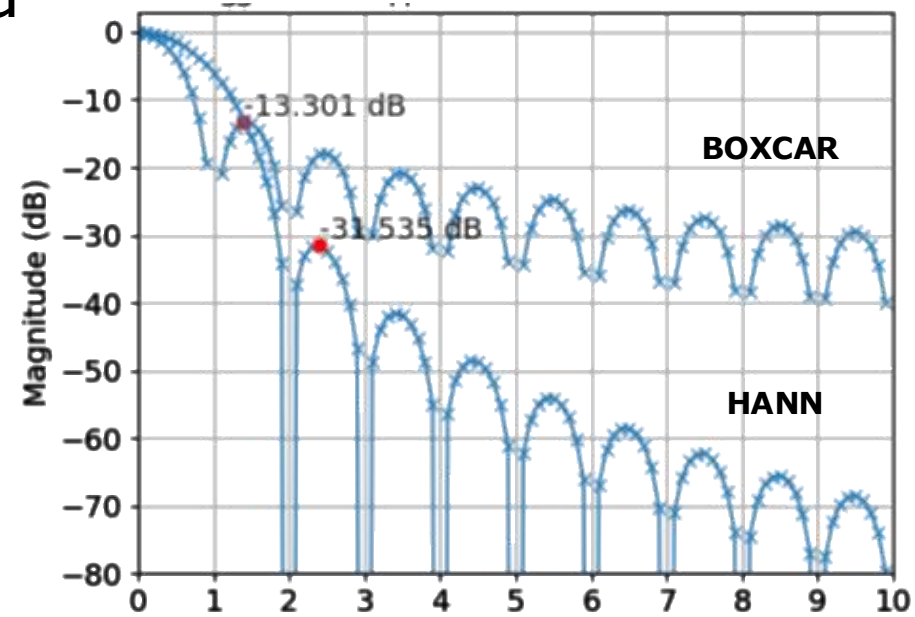


Esercizio: trasformata di finestre comuni

■ Esistono differenti funzioni di finestrata

- ✓ Ciascuna ha i propri valori di
 - Larghezza del lobo principale
 - Altezza del lobo laterale
 - Valore di roll-off

compromesso
trade-off { Al ridursi dell'altezza del lobo laterale
aumenta la larghezza del lobo principale
e/o il roll-off

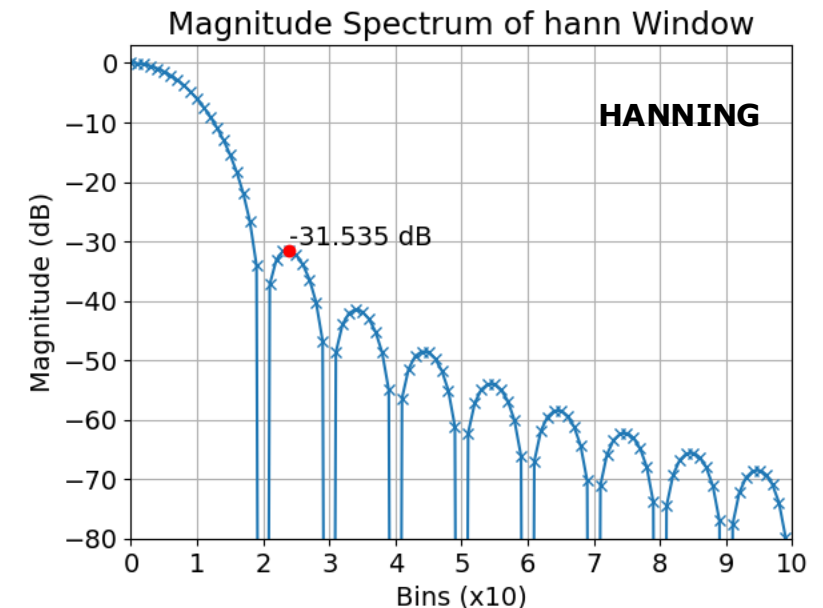


- ## ■ Calcolare i valori di larghezza del lobo principale e altezza del lobo laterale per alcune finestre comuni: `scipy.signal.get_window`
- ✓ Rettangolare (boxcar), Triangolare (triang), Hamming (hamming), Hanning (hann), Blackman (blackman), Bartlett (bartlett)

Esercizio: trasformata di finestre comuni

- Calcolare i valori di larghezza del lobo principale e altezza del lobo laterale per alcune finestre comuni: `scipy.signal.get_window`

```
# Parameters and window generation
N, k = 256, 10 # Number of points in the window, and zero padding ratio
window = get_window('hann', N)
# Compute Fourier transform
fft_window = rfft(window, n=k*N) # Zero-padding to improve frequency resolution
fft_magnitude = np.abs(fft_window) # Center the FFT around zero
nfreqs = np.arange(len(fft_magnitude)) # Bin axis
fft_magnitude /= np.max(fft_magnitude)
# Detect zeros
zeros = np.isclose(fft_magnitude, np.zeros_like(fft_magnitude))
zeropos = np.where(zeros)[0]
n = zeropos[0]
print(f"Main lobe width: {2*n/k} bin")
# Detect peaks after main lobe
peaks, _ = find_peaks(20 * np.log10(fft_magnitude[zeropos[0]:]))
n = zeropos[0] + peaks[0]
x, y = nfreqs[n], 20 * np.log10(fft_magnitude[n])
print(f"Side lobe height: {y:.3f} dB")
```

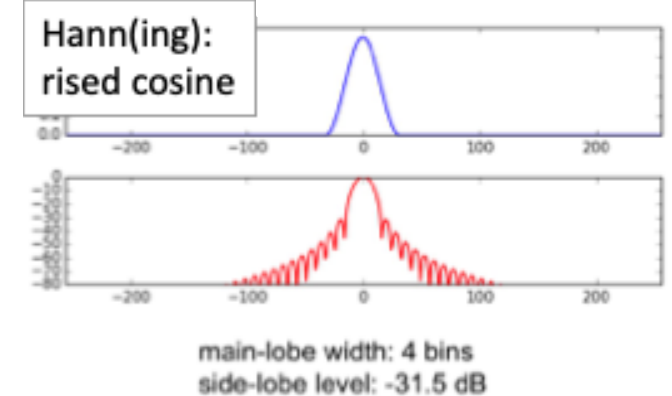
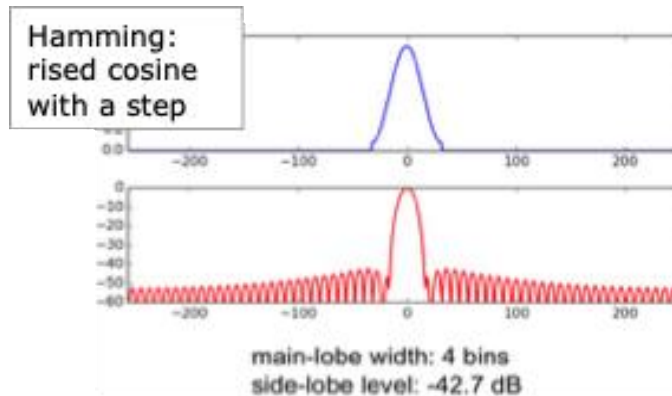
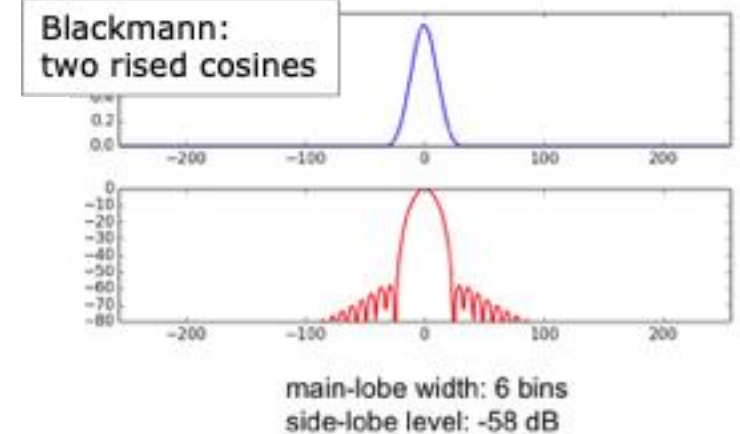
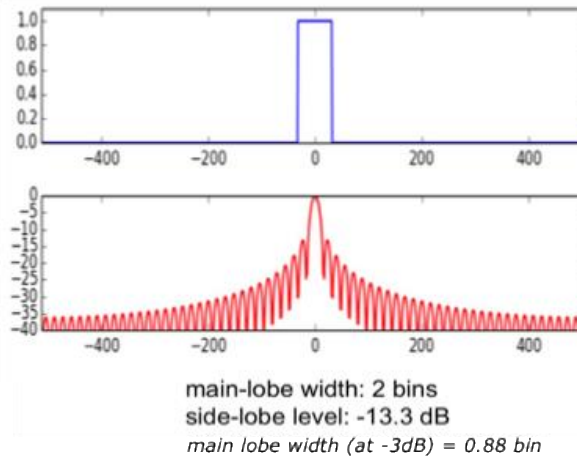


Artefatti di finestre differenti

- Esistono differenti funzioni di finestrazione

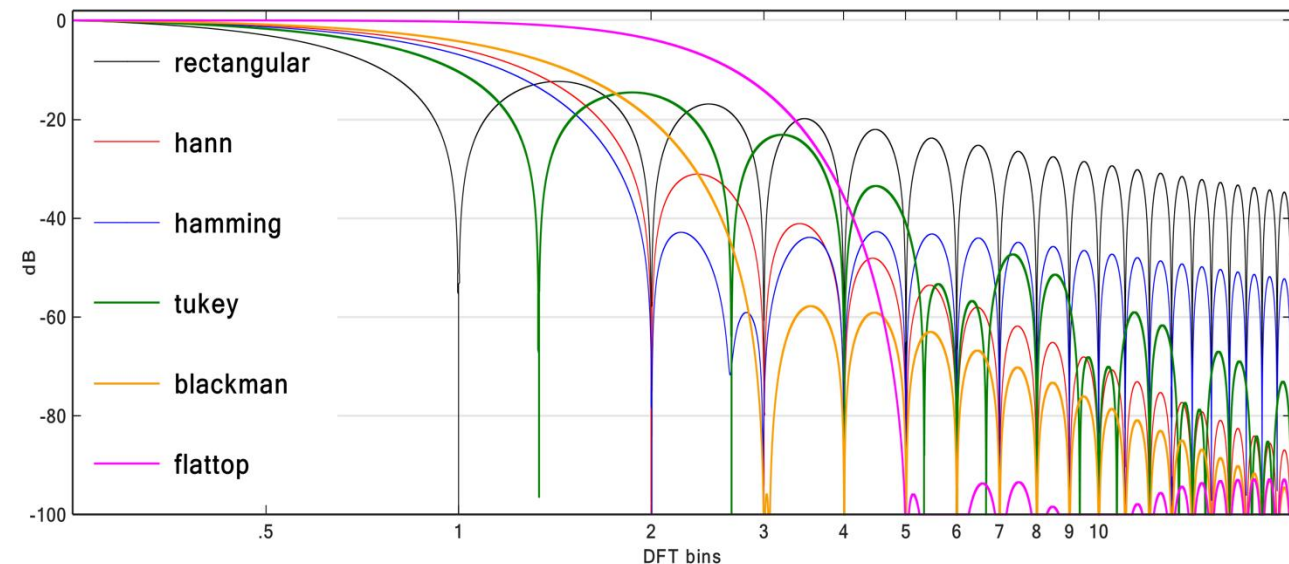
- Trade-off delle proprietà

1. Minor larghezza del lobo principale => migliore discriminazione in frequenza
2. Minor altezza e/o maggiore roll-off lobi laterali => minore rumore nelle bande adiacenti



Artefatti di finestre differenti (bis)

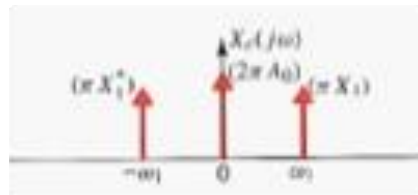
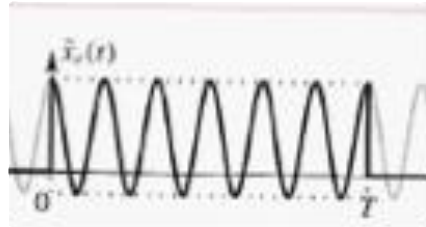
- La larghezza della campana principale causa *smearing*
 - ✓ Influisce sulla capacità di **discriminare** componenti vicine in frequenza (risoluzione dell'asse delle x)
 - ✓ Quando una sinusoide cade nel bin 'k-esimo' il suo contributo/presenza è "spalmato" anche nei bin adiacenti (due, tre, quattro ...)
- L'altezza dei lobi laterali causa leakage (rumore)
 - ✓ Influisce sulla capacità di **stimare** l'intensità di frequenze "lontane" (risoluzione dell'asse delle y)
 - ✓ Una sinusoide che "cade" nel bin 'k-esimo' causa un rumore significativo anche in bin molto lontani



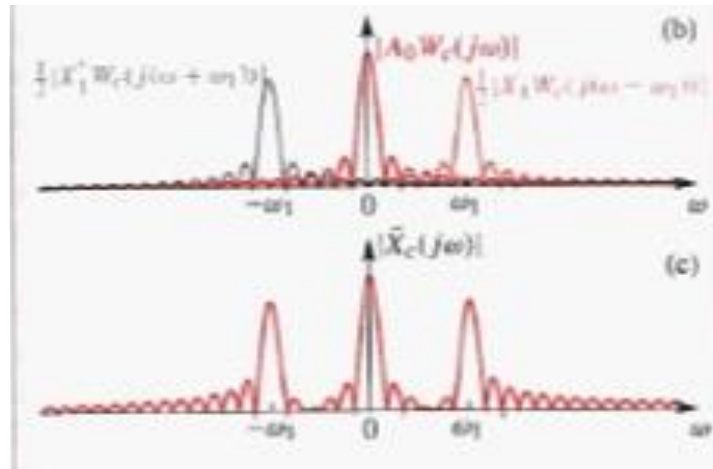
Larghezza del lobo principale

- L'utilizzo di una finestra con lobo principale largo riduce sì il rumore (migliora la stima delle intensità) ma riduce anche la capacità di discriminare frequenza adiacenti

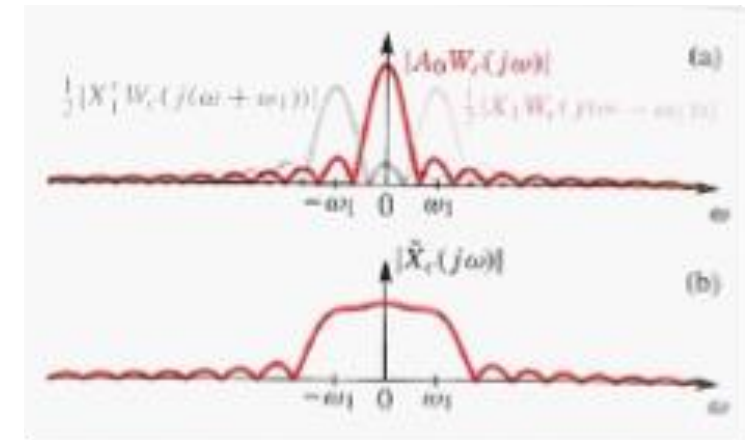
SEGNALE CONTINUO



LOBO CENTRALE STRETTO



LOBO CENTRALE LARGO



Il contributo di frequenze adiacenti viene fuso, sembra rumore a banda larga invece che componenti tonali distinte

Larghezza del lobo principale

- Cosa determina la larghezza del lobo principale?
- 1. Tipo di finestra
- 2. Larghezza / durata della finestra T (o L in campioni)
 - ✓ La larghezza del lobo principale è definita in $\text{bin} = \Delta F$
 - ✓ Più è lunga la porzione del segnale che estraggo migliore è la capacità di discriminare le frequenze
 - ✓ Ragionamento per assurdo
 - $L = 1$, non posso dividere lo spettro (ho un campione solo)
 - $L = 2$, posso dividere lo spettro in basse/alte frequenza (due porzioni)
 - $L = M \Rightarrow$ ho M punti in frequenza
- **Purtroppo, se il segnale è tempo variante, più allungo la finestra più rischio che cambi**

IMPORTANTE

$$\Delta F = \frac{F_s}{L}$$

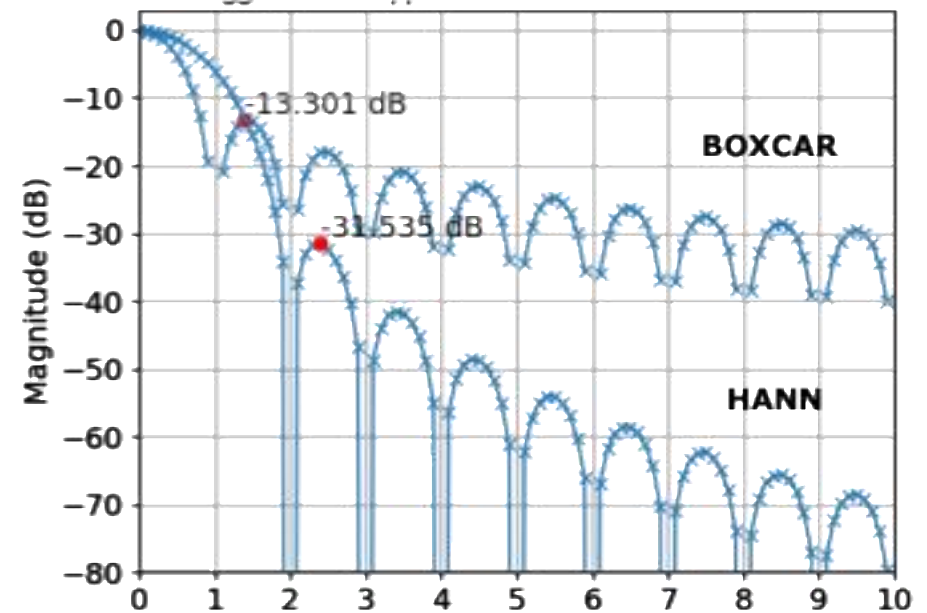
*Risoluzione in
frequenza teorica*

IMPORTANTE

Risoluzione in frequenza effettiva

- In virtù della finestrazione la risoluzione in frequenza EFFETTIVA ΔF_e è data dalla larghezza del lobo principale della finestra
 - ✓ Definita la larghezza del lobo principale b_{ML} in bin, dove la larghezza del bin è fatta coincidere con la risoluzione in frequenza teorica ΔF
 - ✓ La risoluzione in frequenza effettiva è definita come

$$\Delta F_e = \Delta F \cdot b_{ML} = \frac{F_s}{L} \cdot \Delta f_{ML}$$



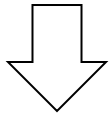
Risoluzione in frequenza

- Teorica: F_s/L
 - ✓ Data dal numero di campioni L su cui si calcola la DFT
- Effettiva: $F_s/L * b_{ML}$
 - ✓ Peggioramento della risoluzione teorica a causa dell'effetto della finestrazione
- Apparente: F_s/N_{fft}
 - ✓ Modificabile "a piacere" campionando in modo più o meno fitto la DTFT che è continua (e periodica), da cui si deriva la DFT

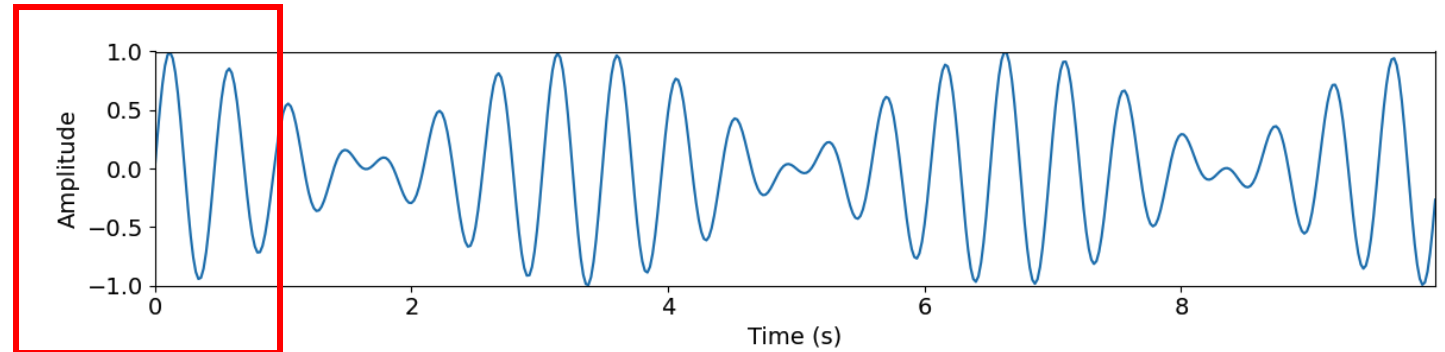
Esempio – risoluzione in frequenza

- Somma di due toni a 2 e 2.3 Hz (con $F_s = 50$)
- Analisi di 1s senza ($N=50$) e con zero padding x 10 ($N=500$)
 - ✓ Le due frequenze non si riescono a discriminare ($dF = F_s/N \Rightarrow 1, 1$ Hz)

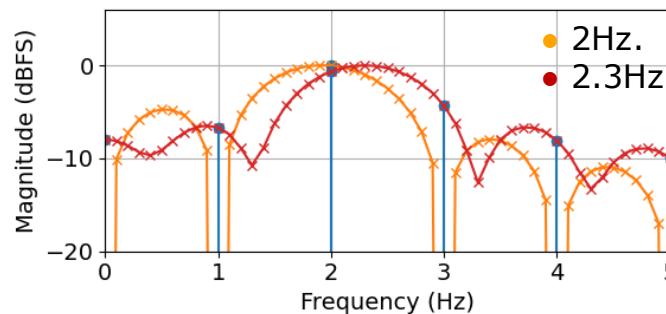
✓ Perché



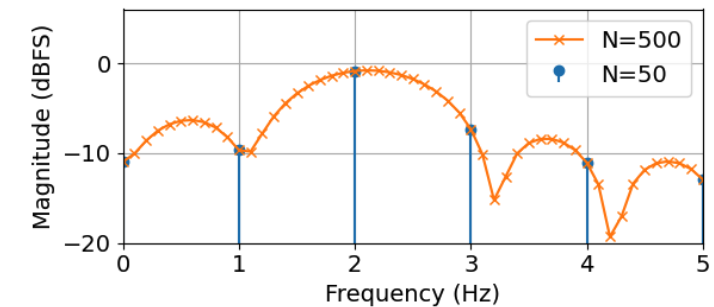
Main lobe width
finestra rettangolare
2 bin $\Rightarrow$ 2Hz



Analisi DFT
delle singole frequenze
con $N=500$



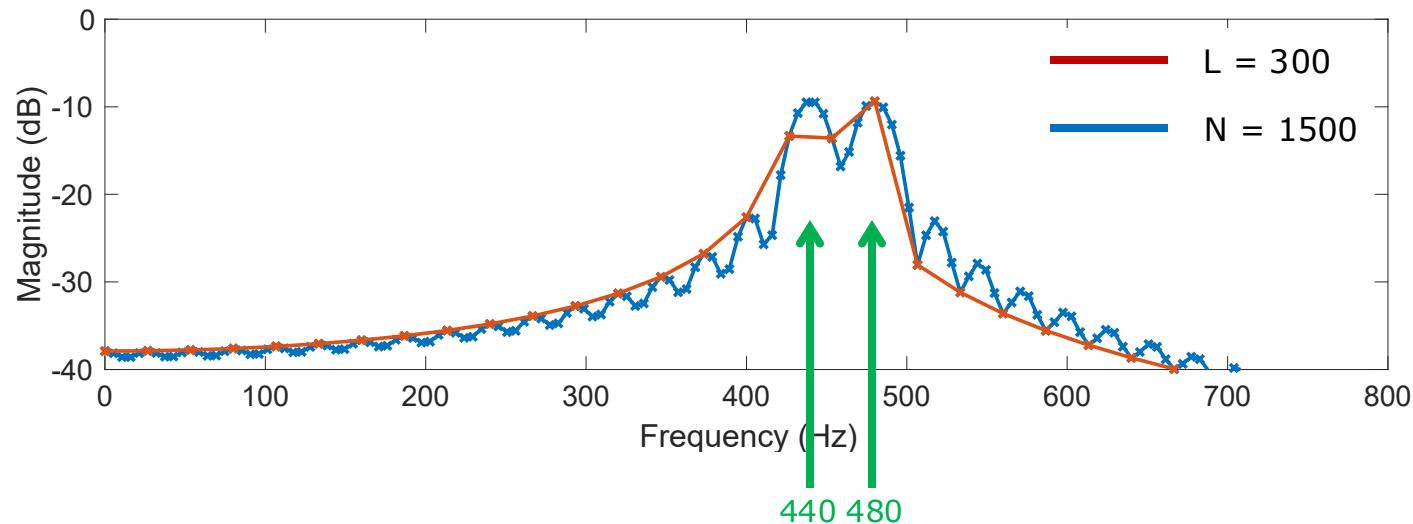
Analisi DFT
somma



Reference: <https://dspillustrations.com/pages/posts/misc/spectral-leakage-zero-padding-and-frequency-resolution.html#Frequency-Resolution>

Zero-padding e risoluzione in frequenza

- Lo zero-padding è un ARTIFICIO aggiunto DOPO il troncamento
- Mi permette di campionare lo spettro continuo in più punti
- Ma non riduce la larghezza del lobo principale



E "seguirlo" meglio

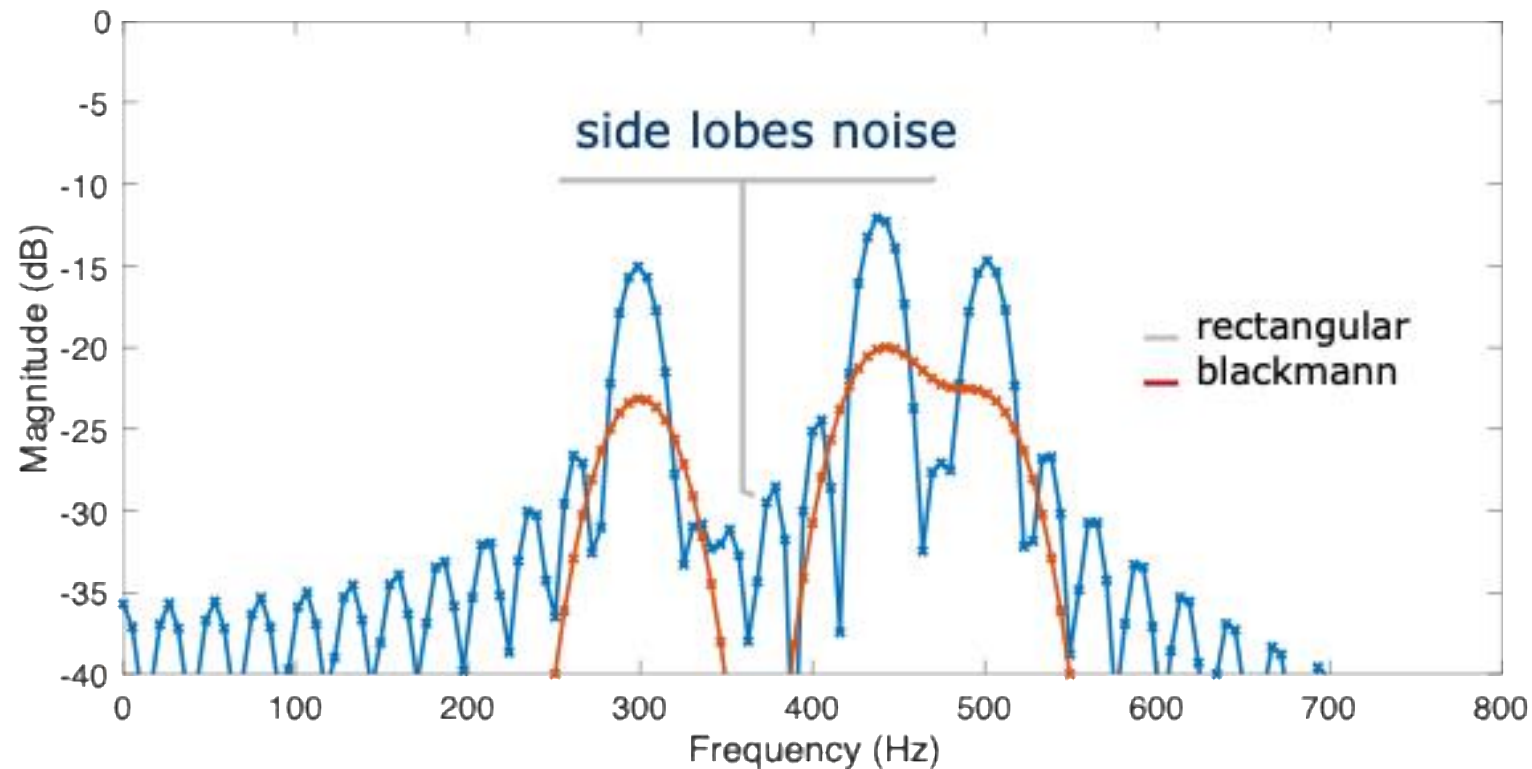
x: somma di due sinusoidi a 440 e 480 Hz

w: finestra di 300 campioni con $F_s = 8 \text{ kHz} \Rightarrow dF = 8000/300 = 26.67 \text{ Hz}$

N: 1500, aggiunti 1300 zeri $dF' = 8000/1500 = 5,3 \text{ Hz}$

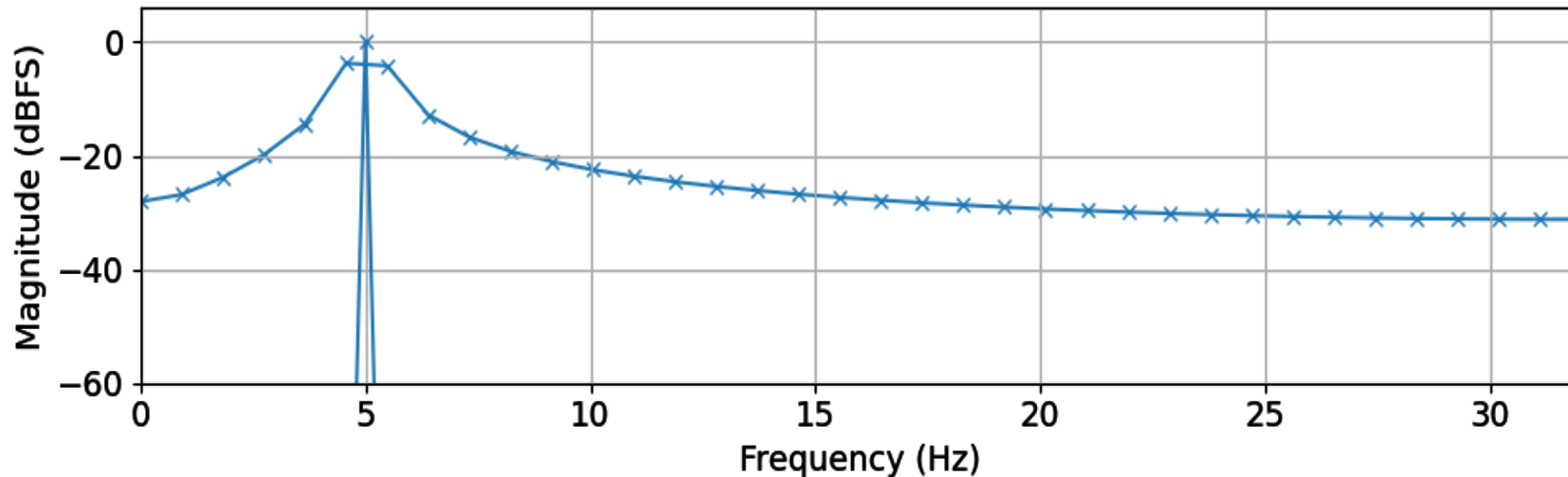
Effetto del tipo di finestra

- x: tre sinusoidi a 300, 440 e 500 Hz ($F_s = 8$ kHz)
- w: rettangolare -13 dB (blu) e blackmann (rossa) -58 dB



Spiegazione esempio iniziale

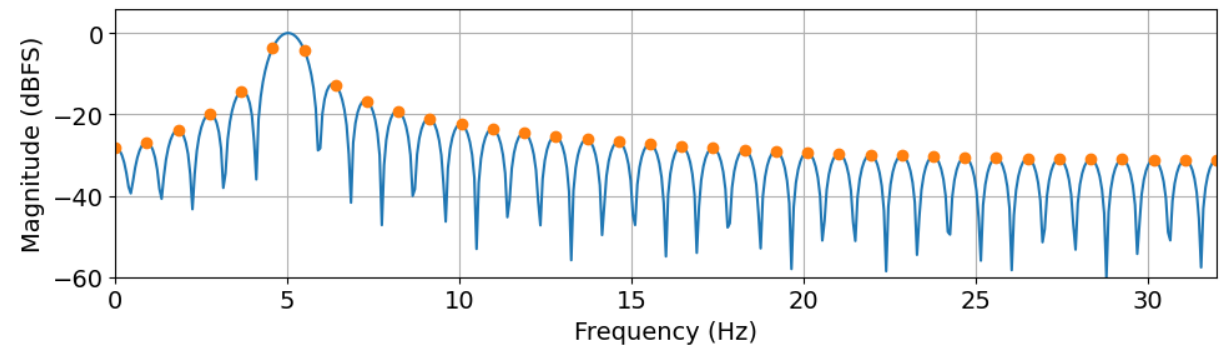
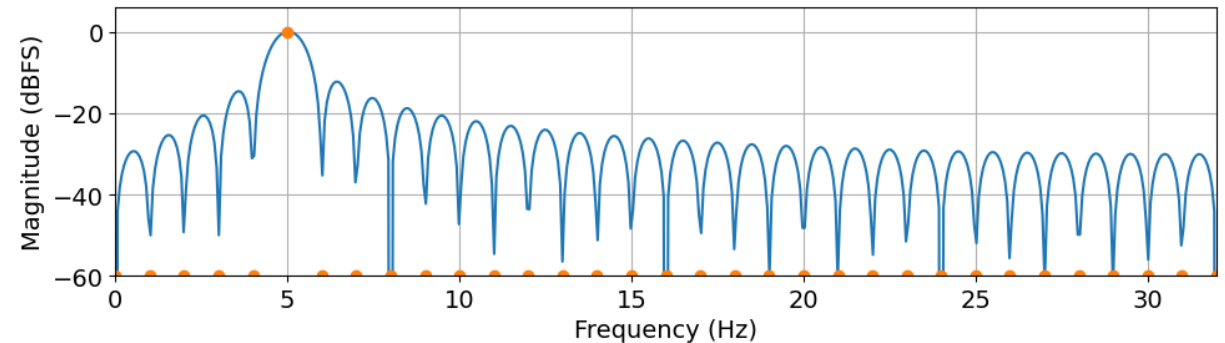
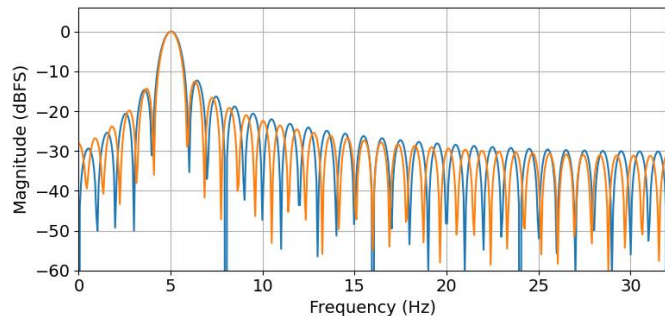
- Inizialmente avevamo considerato un tono da 5 Hz con $F_s=64$ analizzato su una finestra di 1s e 1.1s



- Domanda:
 - ✓ Come spieghereste adesso questo risultato?

Spiegazione esempio iniziale

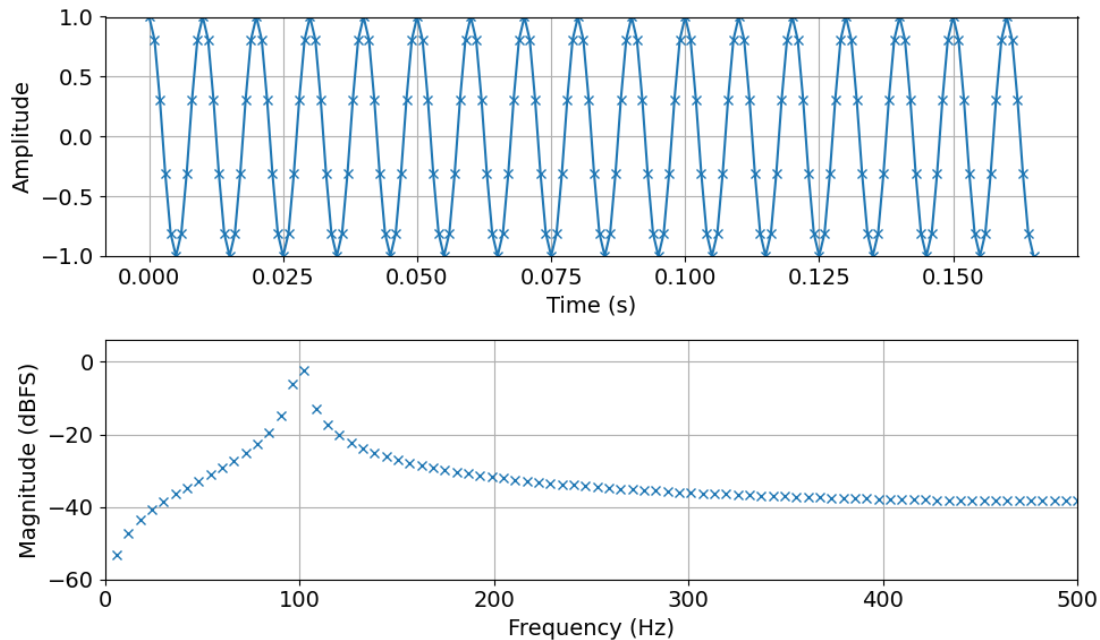
- Inizialmente avevamo considerato un tono da 5 Hz con $F_s=64$ analizzato su una finestra di 1s e 1.1s
 - ✓ La vera differenza è data soprattutto dal diverso campionamento della DTFT



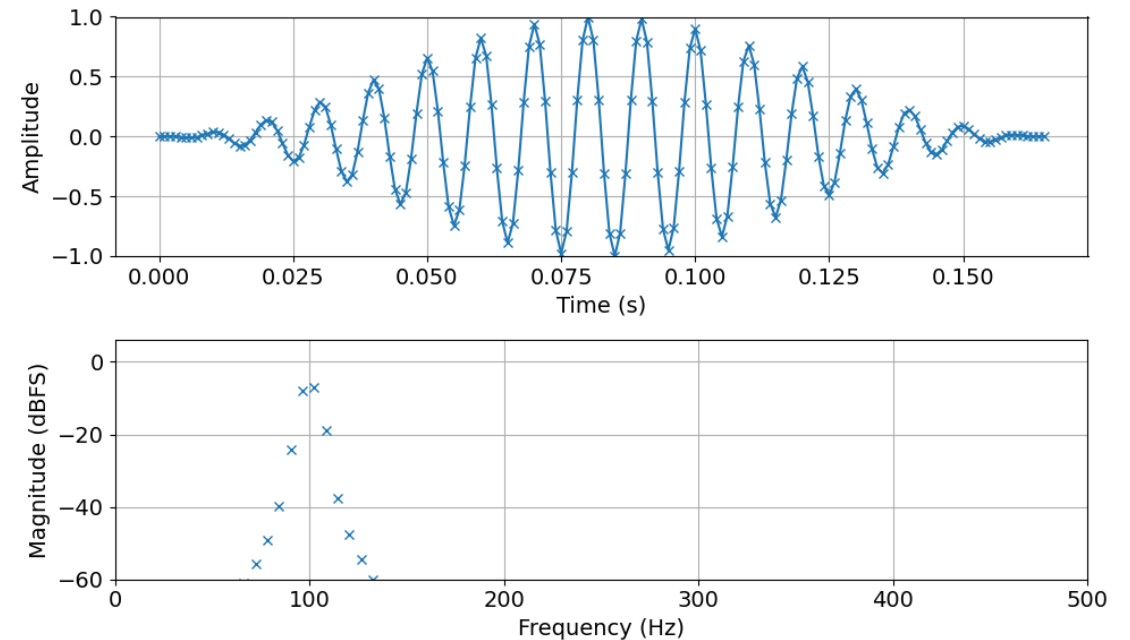
Ascoltare lo spettro del segnale troncato

- Come suona la periodicizzazione di un segnale audio troncato se applichiamo la finestrazione?
- Esempio: senoide a 100 Hz ed F_s 1000

Finestra Rettangolare



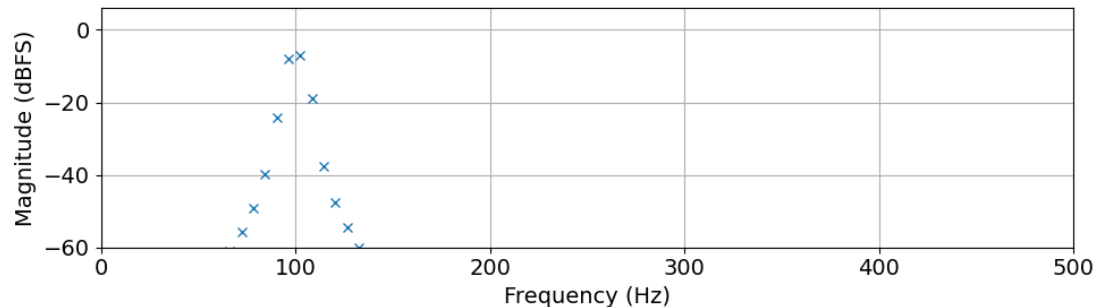
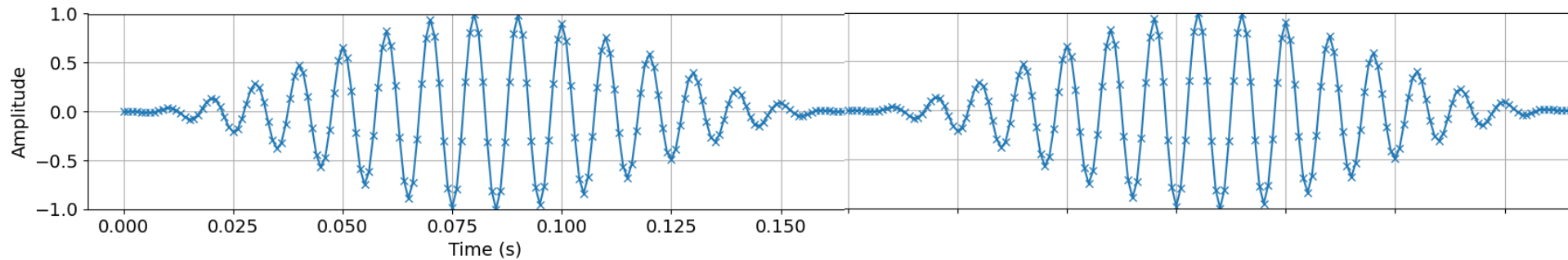
Finestra Hann



Ascoltare lo spettro del segnale troncato

- Come suona la periodicizzazione di un segnale audio troncato se applichiamo la finestrata?
- Esempio: senoide a 100 Hz ed F_s 1000

Finestra Hann



La finestrata "morbida" elimina la discontinuità ma introduce una modulazione

TRASFORMATA DI FOURIER *DISCRETA*

- Tipologie di Trasformate di Fourier
 - ✓ Serie, tempo continuo, tempo discreto, tempo e frequenza discreti
- Python – Trasformata di Fourier Discreta – FFT
- Finestratura ed effetto sulla trasformata
- ➔ ■ **ATTIVITA' – Dual Tone Multi Frequency**

Esercizio - DTMF

■ Dual Tone Multi Frequency

- ✓ Digitando un numero al telefono vengono generati due toni secondo la tabella
- ✓ Le frequenze sono state scelte in modo che le armoniche e le loro intermodulazioni non generino alterazioni (segnali) rilevanti

1 ↗	2 ↗	3 ↗	A ↗	697 Hz
4 ↗	5 ↗	6 ↗	B ↗	770 Hz
7 ↗	8 ↗	9 ↗	C ↗	852 Hz
* ↗	0 ↗	# ↗	D ↗	941 Hz
1209 Hz	1336 Hz	1477 Hz	1633 Hz	

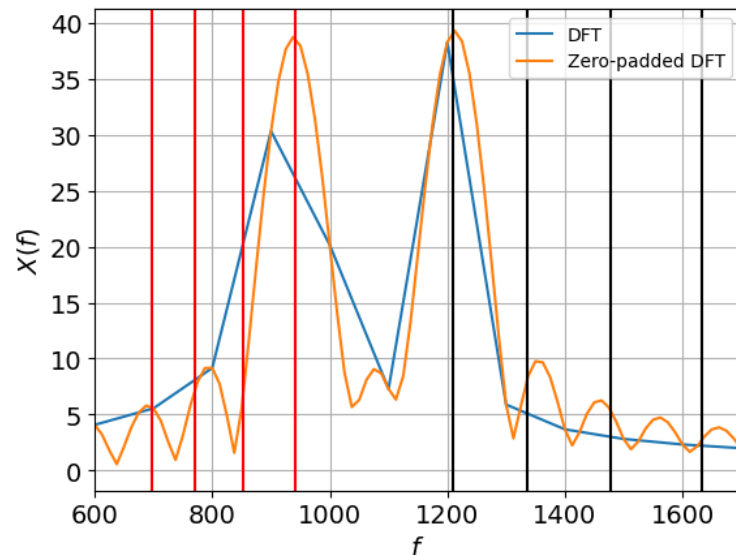
■ Identificazione

- ✓ Inizialmente veniva utilizzato un banco di filtri
- ✓ Attualmente si utilizza l'algoritmo di Goertzel (utile per identificare singoli toni)
- ✓ Noi utilizzeremo la STFT

Esercizio - DTMF

- Definire una funzione che, data una cifra ritorni il suono corrispondente
- Utilizzare la DFT per identificare la cifra dal segnale audio

1 ↗	2 ↗	3 ↗	A ↗	697 Hz
4 ↗	5 ↗	6 ↗	B ↗	770 Hz
7 ↗	8 ↗	9 ↗	C ↗	852 Hz
* ↗	0 ↗	# ↗	D ↗	941 Hz
1209 Hz	1336 Hz	1477 Hz	1633 Hz	



Suggerimenti

- utilizzare toni della durata di 0.1s
(valutare l'efficacia dell'algoritmo per durate inferiori)
- utilizzare lo ZP per ridurre dF quando utile / necessario
- considerare che i picchi si trovano sulle frequenze campionate, non sono quelle esatte della tabella

Esercizio – DTMF con rumore

- Valutare l'efficacia dell'algoritmo nel caso di audio con disturbo
 - ✓ rumore bianco: non ha componenti tonali
 - ✓ eventuali altri suoni di sottofondo
musica, dialoghi, suoni ambientali
- Identificare oltre quale livello di energia del segnale di disturbo / lunghezza del tono l'algoritmo non è più affidabile

1 ↗	2 ↗	3 ↗	A ↗	697 Hz
4 ↗	5 ↗	6 ↗	B ↗	770 Hz
7 ↗	8 ↗	9 ↗	C ↗	852 Hz
* ↗	0 ↗	# ↗	D ↗	941 Hz
1209 Hz	1336 Hz	1477 Hz	1633 Hz	

```
sigma: 8.90 => 0.234.6.8.A..D#*  
sigma: 9.00 => 01.3.5..89..C.#*  
sigma: 9.10 => .12.....A..D..  
sigma: 9.20 => .1.....7..AB.D#*  
sigma: 9.30 => 0....567..AB.D..  
sigma: 9.40 => .12.4.678.AB..#.  
sigma: 9.50 => 0..3...78.A...#.  
sigma: 9.60 => 0123.5.7...B..#.  
sigma: 9.70 => 0.2.4..78.AB....  
sigma: 9.80 => ..2.....B...*  
sigma: 9.90 => 01..4.67.9.B..#.
```

